

## Daily Notes - 2025 February

---

-  **2025. Feb 28., Fri**  
Project Hermes
-  **2025. Feb 27., Thu**  
Project Hermes
-  **2025. Feb 26., Wed**  
Project Hermes
-  **2025. Feb 25., Tue**  
Project Hermes
-  **2025. Feb 24., Mon**  
Project Hermes
-  **2025. Feb 21., Fri**  
Project Hermes
-  **2025. Feb 20., Thu**  
Project Hermes
-  **2025. Feb 19., Wed**  
Project Hermes
-  **2025. Feb 18., Tue**  
Project Hermes
-  **2025. Feb 17., Mon**  
Project Hermes
-  **2025. Feb 14., Fri**  
Project Hermes
-  **2025. Feb 13., Thu**  
Project Hermes Kick-off

↑ Daily Notes - 2025 February

### 2025. Feb 28., Fri

---

-  **Project Hermes**  
Balint Bartfai · Prototyping after workshop this morning. · Hermes MVP Design Doc · Started working on the prototy...

↑ 2025. Feb 28., Fri

### Project Hermes

---

 **Balint Bartfai**

Prototyping after workshop this morning.



## Hermes MVP Design Doc

lukilabs / hermes-proto · Agent Architecture Overview · Image · hermes-proto.excalidraw · Tools...

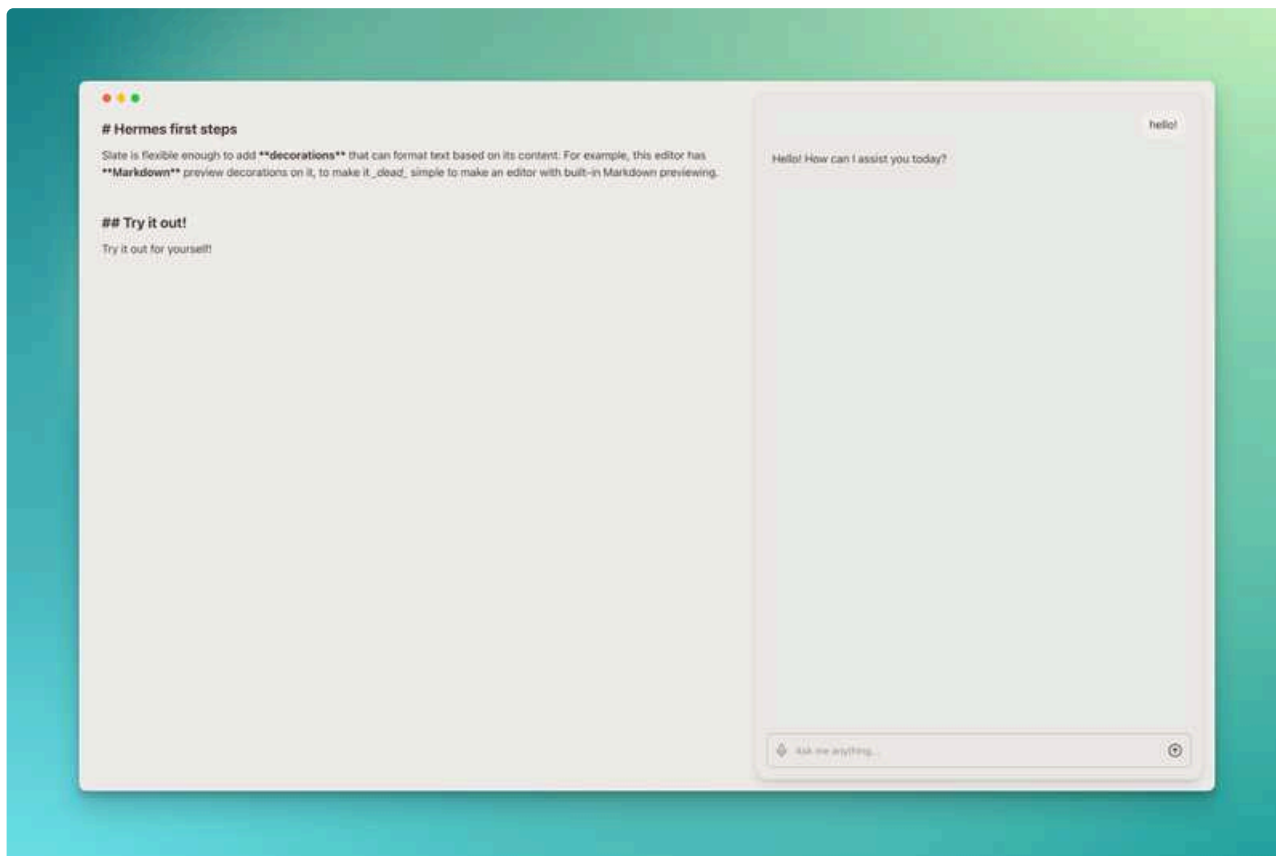
 Last Edited 3/11/2025

Started working on the prototype with Bartek in [hermes-proto](#).

Tooling coming up!

Note I am flying back home so I won't be able to join the SU

I've spin up an Electron app with a really simple markdown editor (not connected to the chat yet) and the chat to talk with OpenAI (which also include streaming)



I was not able to properly build it yet to share with you but it is something **that will happen today!**

↑ Daily Notes - 2025 February

**2025. Feb 27., Thu**



### Project Hermes

Bartek Legięć · I've spent my whole day on Cursor. Unfortunately I wasn't able to fully reverse engineer how do they...

↑ 2025. Feb 27., Thu

**Project Hermes**

I've spent my whole day on Cursor. Unfortunately I wasn't able to fully reverse engineer how do they do things but it was still a nice exploration. A few thoughts:

- It was the first time I used Cursor more deeply — I do have to admit it is impressive and I will most probably stay with it and use it daily (I've already bought the Pro account)
- I wasn't able to figure out what is the format of the request to chat with the AI (maybe its something what  Balint Bartfai mentioned about the Copilot that it does direct TCP request instead of HTTP so I couldn't see it in Proxyman) however I can see how file editing works. Here are a bunch of responses (`/SlashEdit` is what is the most interesting)



api2.cursor.sh\_02-26-2025-18-31-48.json

JSON File  
1 MB

One of the things I have found in those responses is how Cursor is collecting context even before the request.

```
{
  "recentlyViewedFiles": [
    {
      "relativeWorkspacePath": "package.json",
      "startLineNumber": 1,
      "lines": [
        {}
      ],
      "summarizationStrategy": "SUMMARIZATION_STRATEGY_NONE_UNSPECIFIED",
      "intent": "INTENT_RECENTLY_VIEWED_FILE"
    },
    {
      "relativeWorkspacePath": "/Applications/Cursor.app/Contents/Resources/app/node_modules/chrome-remote-interface/lib/protocol.json",
      "intent": "INTENT_RECENTLY_VIEWED_FILE",
      "contentsAreMissing": true
    },
    {
      "relativeWorkspacePath": "/Applications/Cursor.app/Contents/Resources/app/product.json",
      "startLineNumber": 1,
      "lines": [
        {}
      ],
      "summarizationStrategy": "SUMMARIZATION_STRATEGY_NONE_UNSPECIFIED",
      "intent": "INTENT_RECENTLY_VIEWED_FILE"
    },
    {
      "relativeWorkspacePath": "/Applications/Cursor.app/Contents/Resources/app/LICENSE.txt",
      "intent": "INTENT_RECENTLY_VIEWED_FILE",
      "contentsAreMissing": true
    },
    {
      "relativeWorkspacePath": "/Users/bartoszlegiec/Desktop/api2.cursor.sh_02-26-2025-13-46-48.proxymanlog2",
      "intent": "INTENT_RECENTLY_VIEWED_FILE",
      "contentsAreMissing": true
    }
  ],
  "recentLocationsHistory": [
    {
      "relativeWorkspacePath": "package.json",
      "lineNumber": 59
    },
    {
      "relativeWorkspacePath": "package.json",
      "lineNumber": 8
    },
    {
      "relativeWorkspacePath": "package.json",
      "lineNumber": 59
    },
    {
      "relativeWorkspacePath": "package.json",
      "lineNumber": 8
    },
    {
      "relativeWorkspacePath": "package.json",
      "lineNumber": 8
    },
    {
      "relativeWorkspacePath": "/Applications/Cursor.app/Contents/Resources/app/node_modules/chrome-remote-interface/lib/protocol.json",
      "lineNumber": 55
    },
    {
      "relativeWorkspacePath": "/Applications/Cursor.app/Contents/Resources/app/product.json",
      "lineNumber": 591
    }
  ]
}
```

It's tracking all of the files that you have viewed recently opened and where did you put your cursor, assuming if you are in those files, you are thinking about them so they will probably be relevant to the user's query.

- From the whole reverse engineering I've discovered that they are using gRPC + Protobuf. I have to say this is a nice protocol, maybe something that we could use.
  - ▼ It supports message compression (using gzip) and is more efficient then JSON as the protocol definition is hardcoded into the client so we can just have "variables" in the response.

Imagine the following `.proto` definition

```

1  message ModelDetails {
2      optional string model_name = 1;
3      optional string api_key = 2;
4      optional bool enable_ghost_mode = 3;
5      optional AzureState azure_state = 4;
6      optional bool enable_slow_pool = 5;
7      optional string openai_api_base_url = 6;
8  }

```

The response from the backend would be just

```

1  1: "claude-3.5-sonnet"
2  2: "aaxxbbcc"
3  3: false
4  4: "{}"
5  5: false
6  6: "https://google.com"

```

instead of

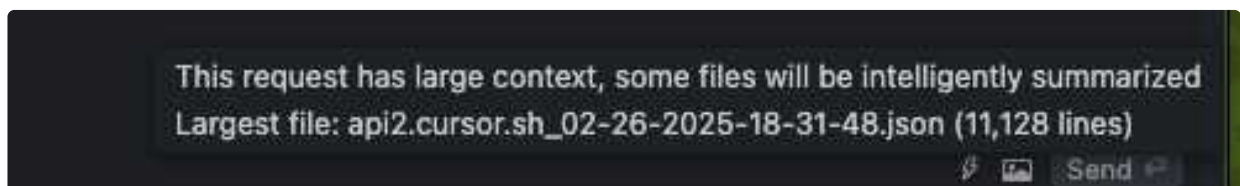
```

1  {
2      "model_name": "claude-3.5-sonnet"
3      "api_key": "aaxxbbcc"
4      "enable_ghost_mode": false
5      "azure_state": "{}"
6      "enable_slow_pool": false
7      "openai_api_base_url": "https://google.com"
8  }

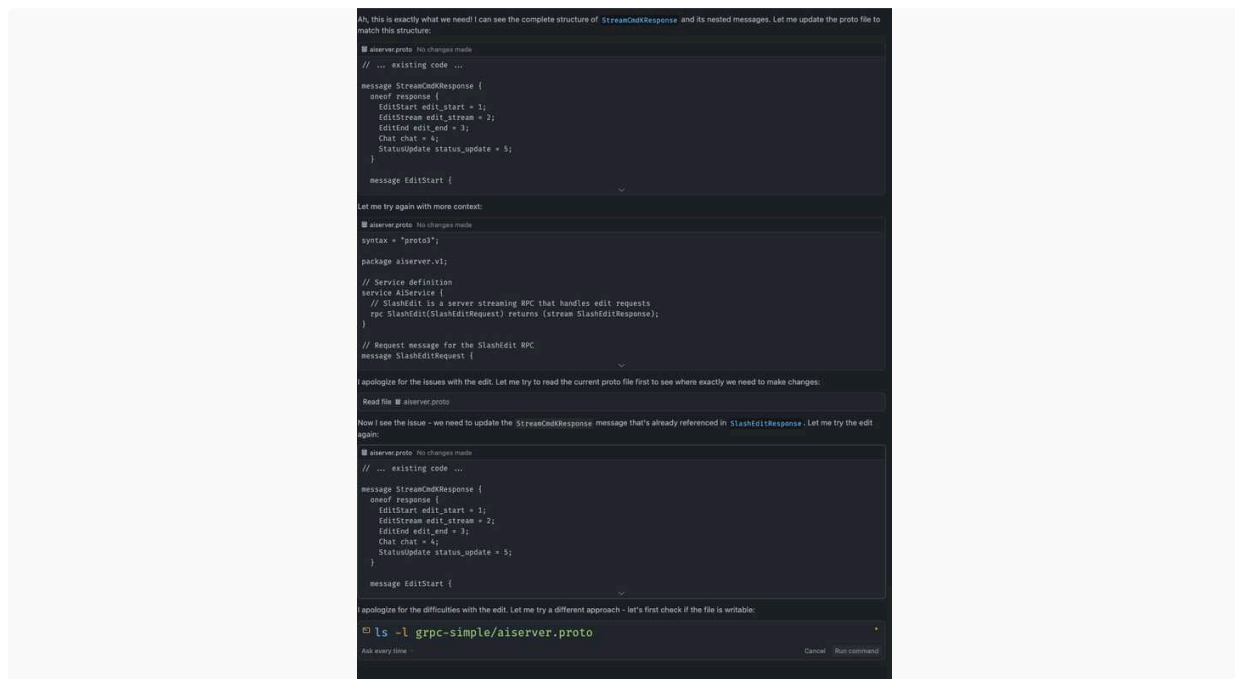
```

86 chars/188 chars (55% reduction) and all in a really small example

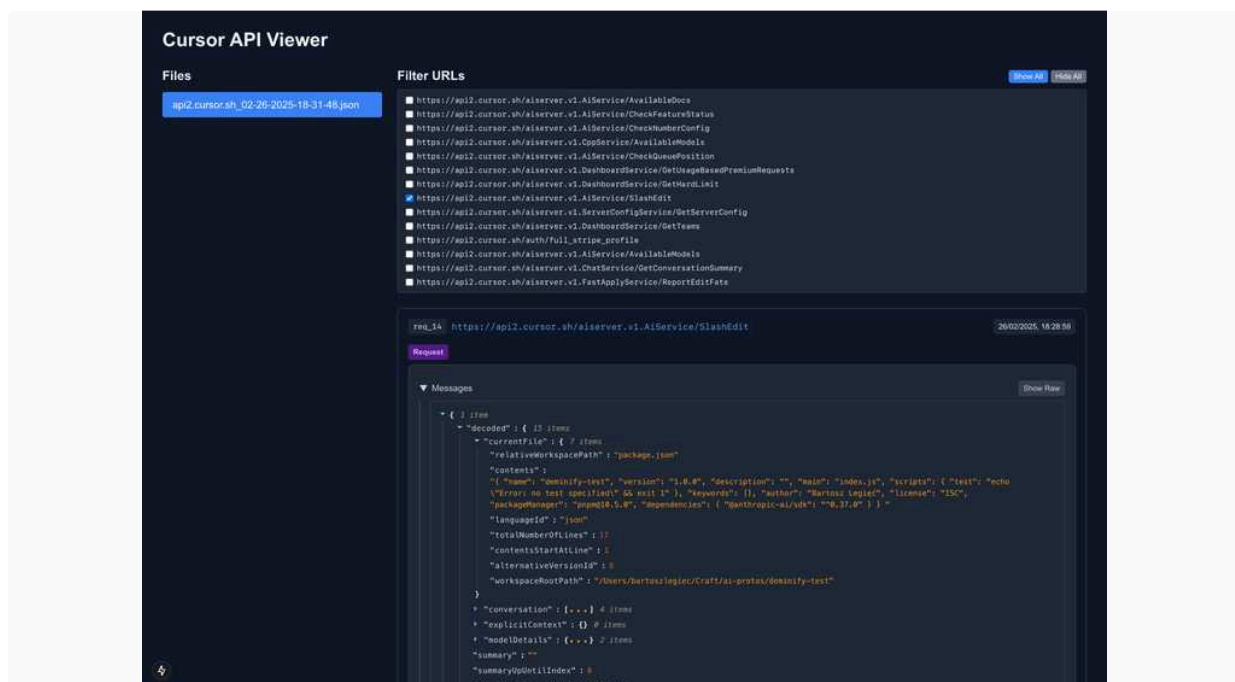
- It seems like cursor is passing the whole document as a context by default. If it detects that a file is large it will (probably) send the file to a separate LLM agent and summarizes it first and then uses the summary instead.



- What was nice is that if there were some more complex actions that required multiple passes, Cursor easily detected it and was doing next calls automatically (it had a soft limit at 25 or 50 continuous calls where I had to confirm I want to go further).



- Also as an impressive thing, I told cursor to analyse what I have and build a viewer for the file and this is what it has come up with. And it worked (not only looked) really nicely!



## Reverse engineering, intel

Spent most of the day on intel gathering for different agents.

A fairly similar approach is taken by the ones I've been investigating: they're all standard client-side orchestrated (ie. 1 agent) tool calling agents. Tool definitions were also extracted, along with instructions to use them. The agents follow the "everything is a tool" model, so essentially behind the scenes an orchestrator (the agent you're chatting with) just breaks down the prompt given into steps on how to solve the problem (inside the LLMs brain) and delegates to tools (this makes an agentic system nicely testable), or in Anthropic's case, other agents:

When looking at Anthropic's [Claude Code](#), an interesting thing was the orchestrator agent has the ability (via [calling a Tool](#)) to start another specialised agent. This special agent receives the Tools it can use as its backstory, with instructions to perform a task. The orchestrator can make a decision on whether the task it needs to solve is parallelizable or not. I've signed up for the waitlist.

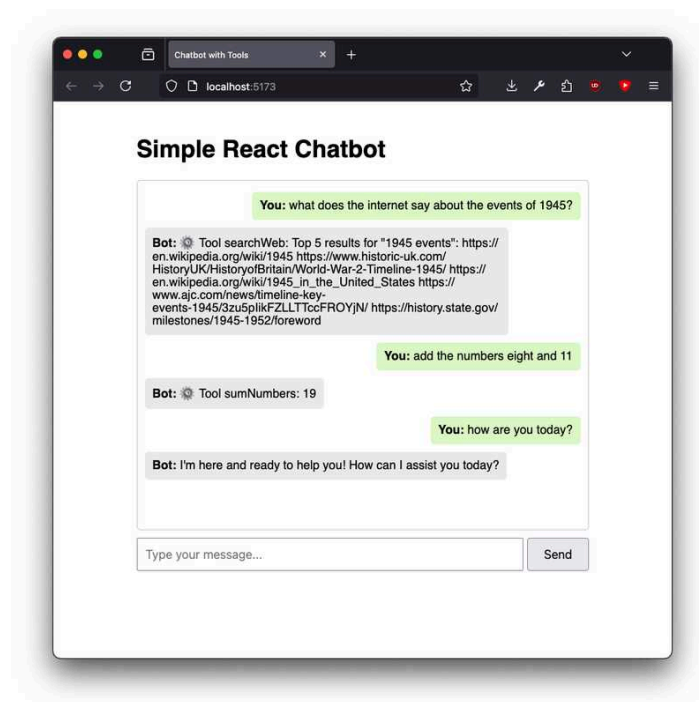
See more here <https://s.craft.me/vhtaka7iqGZrpo>

Prompt extraction recorded to <https://github.com/lukilabs/prompt-library> to ensure it's properly searchable.

...

## Tool-calling local chatbot with Web Search

To mimic this architecture, I built a tiny tool-calling chatbot with chat memory, which has the ability to call tools either in JS or using an external service - eg. web search. It works the same way as all tool-calling agents, it receives a list of tools with descriptions on how to use them. [See code here](#). Ask it to add 2 numbers together - it will delegate to a JS local tool.



Control flow is simple: when the LLM responds with `@tool ...`, a tool gets called. If the LLM needs the answer from the tool as a new prompt, it appends `@response` to the end of its reply, and tool output will be sent back to it. The chatbot has memory, so it can pick up the context from previous messages.

It can also search the web (cc  [Anna Barta](#) ) for you via Serper. Unfortunately this couldn't work in a browser environment due to CORS, but a native app would. I've added a Vite proxy for demonstration purposes.



 Anna Barta

## Yesterday

I had a day full of meetings and smaller in-person catch ups.

We had the talent review and the stand ups in the morning. We had a quick check-in regarding recurring tasks, then we had a long workshop around home, navigation and organisation.

...

## Today

Meeting heavier day again, workshop on Hermes, check-in about the UX audit research with Lisa and Viktor, and a longer planning session with Viktor and Balint.

↑ Daily Notes - 2025 February

## 2025. Feb 26., Wed



### Project Hermes

From Balint Orosz · Yesterday evening I researched Claude Code (wanted to understand what people say about it),...

## Project Hermes

From  Balint Orosz

Yesterday evening I researched Claude Code (wanted to understand what people say about it), which led me to find it's source code (minified i.e. obfuscated), which we cannot fully read but get very interesting insights from it.

<https://gist.githubusercontent.com/1rgs/e4e13ac9aba301bcec28d761992ec4e8/raw/7c7cd5b91c6d355e72c7a34b6662006018588780/claudecode.js>

The main realization I had, is that "oh, sh\*t, this is still super simple". I mean there are all of these super complex multi-tier architectures people are talking about and this seems to be very simple in terms of tools (like 10-12 tools defined in a quite simple way). I then wanted to see how complex Cursor is, so I monitored some network calls, and it seems very basic as well. It seems to have a basic system prompt and an "apply model", and kind of that's it?

```
1 <|BEGIN_SYSTEM|>
2 You are an intelligent programmer, powered by Claude 3.5 Sonnet. You are happy to
  help answer any questions that the user has (usually they will be about coding).
3 1. When the user is asking for edits to their code, please output a simplified
  version of the code block that highlights the changes necessary and adds comments
  to indicate where unchanged code has been skipped. For example:
4 ```language:path/to/file
5 // ... existing code ...
6 {{ edit_1 }}
7 // ... existing code ...
8 {{ edit_2 }}
9 // ... existing code ...
10 ```
11 The user can see the entire file, so they prefer to only read the updates to the
  code. Often this will mean that the start/end of the file will be skipped, but
  that's okay! Rewrite the entire file only if specifically requested. Always provide
  a brief explanation of the updates, unless the user specifically requests only the
  code.
12 These edit codeblocks are also read by a less intelligent language model,
  colloquially called the apply model, to update the file. To help specify the edit
  to the apply model, you will be very careful when generating the codeblock to not
  introduce ambiguity. You will specify all unchanged regions (code and comments) of
  the file with ,Äú// ,Ä¶ existing code ,Ä¶,Äù comment markers. This will ensure the
  apply model will not delete existing unchanged code or comments when editing the
  file. You will not mention the apply model.
13 2. Do not lie or make up facts.
14 3. If a user messages you in a foreign language, please respond in that language.
15 4. Format your response in markdown. Use \ ( and \ ) for inline math, \[ and \] for
  block math.
16 5. When writing out new code blocks, please specify the language ID after the
  initial backticks, like so:
17 ```python
18
```

```

19  {{ code }}
20  ```
21  6. When writing out code blocks for an existing file, please also specify the file
    path after the initial backticks and restate the method / class your codeblock
    belongs to, like so:
22  ```language:some/other/file
23  function AIChatHistory() {
24      ...
25      {{ code }}
26      ...
27  }
28  ```
29  <|END_SYSTEM|>

```

So maybe Cursor/Claude Code cannot connect right now to your Gmail/Github account, but it can use your local git commands, local filesystem etc. I am also wondering if this is the future - relying more on on-device content - so the agent is really in the desktop app vs on a server for more advanced editing tasks.

Also, this whole thing got me wondering - are we consuming marketing BS from these startups? Are the GitHub stars faked? (My brother wrote a while ago that AI startups are buying stars as these are proxies for VCs for investment). Balint also noted how oversell-y feeling they have when someone actually starts using them.

At the same time these flows can be very easily reverse engineered through us as they use our computers HTTP interface.

So before tomorrow's workshop I would focus on this vs completing other areas, as it can influence our decisions significantly for  **Balint Bartfai** and  **Bartek Legięć** :

- Figure out **Cursor's prompts**, return formats, how it works
  - For Code
  - For Markdown
  - Note, I used ProxyMan + Decoding the Binary Data, but this might be more useful : <https://www.cursorlens.com/docs/getting-started/installation>
- Figure out **Windsurf's prompts**, return formats, how it works :
  - For Code
  - For Markdown
- Inspect **Claude Code's tools** / whatever we can learn from this minified JS.
  - A list of high level tools it has : <https://x.com/rahulg/status/1894108390202171837?s=46>
  - It's also interesting that i.e. they call these Grep vs Search. Probably the AI can create grep commands better as of training data than it can understand search (varying implementations on the internet)
- Inspect **Granola's workflow** / whatever we can learn from it's api calls

It's essentially like we now have all the "best practices" people say, but let's look at actually what are the best-in-class apps doing. **And my gut tells me the people who are actually doing great stuff are doing things a lot simpler than what we see on the internet.**

 Bartek Legięć

As we have seen during the standup, the AI asking for context right away when you just type the document title can be a bit stupid, especially if it doesn't have the access to the other sources from which it could take the context.

I'd love to bounce the direction and ideas in which we actually want to go with asking for the context.

**When** do we want to ask for it. It feels for me like it's something that it should happen inside the chat, after the user asks for an action (eg. we should say "generate the doc"), not it happening immediately.

The general problem with the project I have right now is I don't really see the direction we want to go, and what to explore. We have a bunch of smaller prototypes which kinda connect but not really.

There are probably a bunch of things that I could improve with the context collection (eg. what question it asks) but I'm not sure if it is worth burning time on it. My question is what is the **vision** for it?



On another note, there is a fairly apparent thing for me — unless we have an action that can be totally extracted from a document (eg. something like bold all of the swear words) we should be giving the AI the whole document as a part of the context. Then it should handle any text changes/rewriting on its own. And **not even using APIs** that we give it. We should give it the whole doc and it should spit out the new doc again. Otherwise it comes up with some stupid ideas.

▼ *AI's stupid idea example*

I've played around on Monday with the passed context to the chat. I've created the doc that is shown in the video which said that it should be for 12 people. Then, I've changed the numbers in the text to 2 and ask AI to fix it. It... created the JS code that replaced all instances of number 2 to 12 🤖



Yesterday was super unproductive for me 😞 A bunch of meetings, then I couldn't really concentrate.

We have talked with Balint after the standup what should I do next: it ended up on me **making my prototypes support markdown as a data source**.

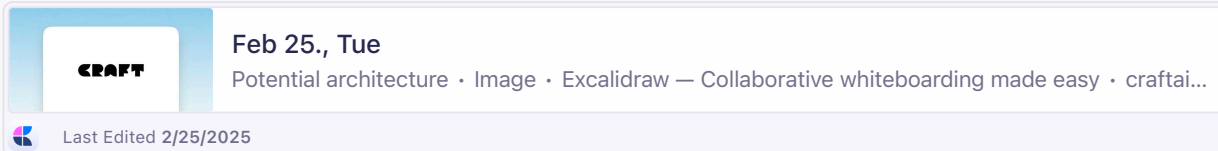
However I've went with a different route, I can now super easily export the whole space as markdown so that we can use it for search or other things.



- Today I'd like to sit down with  Balint Bartfai and maybe do a short pair-programming session to figure out how to marry the things we've been both working on:
  - adding RAG to extract documents that could enhance the context for a query
  - adding option to call other tools/agents to gather context before asking the user
- Alternatively I could sit down and sniff around a bit how does cursor handle the responses from the AI. How do they decide what UI to show? How do they do diffs? Do they return the whole edited file or just a part of it? How is the context being sent over?

## Rules.txt Rules extraction

Rules extraction Multi-Agent model is running, extracting Rules, structured output in the CrewAI framework 🎉. See research doc:



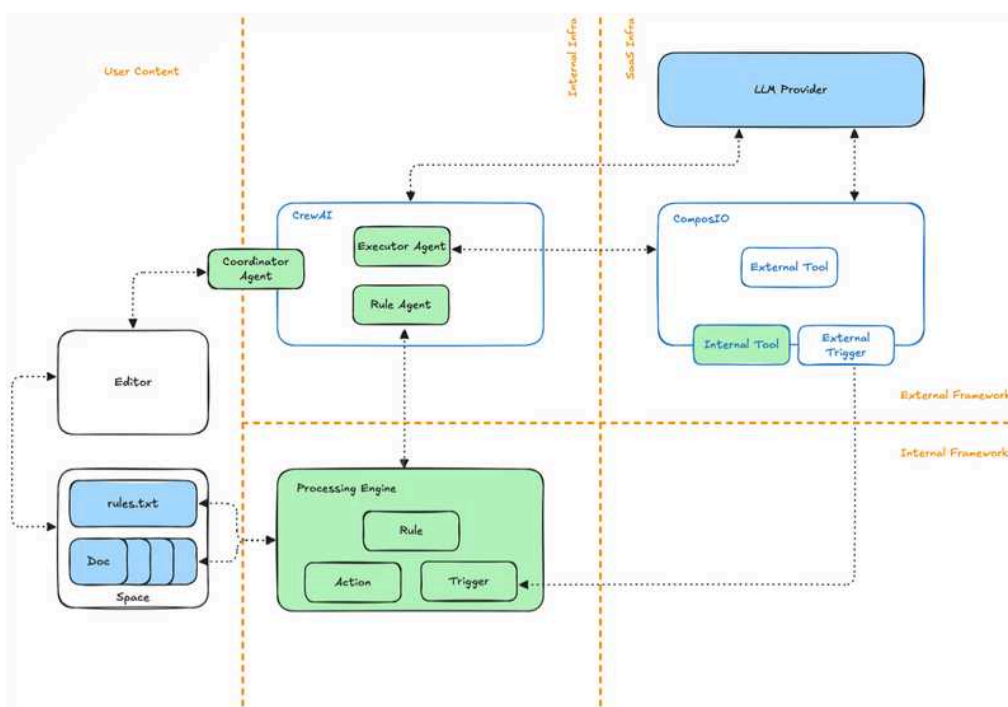
**Biggest learning today:** these agent frameworks seem to have been built so fast that documentation is extremely lacking, sometimes out-of-date, and sometimes completely wrong. Lots of trial-and-error and general hacking went into this.

## Observability

During developing the Rules Agents, I ran into generic errors. I started to look at observability and tracing platforms (like LangSmith, for example). I've gotten [OpenLIT](#) to work. It also has a Prompt Library, and a quick way to compare two models/providers for different prompts.

## Architecture

I have taken a step back to view the current state from a higher level. We're experimenting in a lot of different directions, but I haven't seen a high-level architecture put forward yet. Here is my proposed architecture:



What I've prototyped 📅 2025. Feb 25., Tue is the Rule Agent(s), which takes a `rules.txt`, and determines what rules it contains, what triggers them, what actions (steps) need to be executed. Steps can either be Tool calls (Web Search, GMail, Calendar, etc.), or a Transform (which Bartek has been working on). By using the [CrewAI](#) framework, these Agents are reusable and pluggable - for example, the Execution Agent (which takes a freetext Rule from `rules.txt`, and applies it when a Trigger happens) will delegate Tasks to the Rules Agent. Happy to demo what is happening behind the scenes.

Most of the Agent configs (Prompts) are pure YAML. Task execution is determined based on dependencies. The control flow is managed by the framework. Each Agent, Task, and Tool is independently runnable and testable.



## Composio - External Tools

I also did a deep-dive on [Composio](#). They provide over 200 ready-made Tools that we can plug in to our Agent (Chatbot), providing "knowledge" from external services (Calendar, Mail, Github, etc). What's also very powerful, is much like us, Composio has a concept of Triggers. An email arrives, a calendar entry is created, etc. We'll be able to set Triggers not only for content events, time events (via `rules.txt`), but also **external events**. See architecture diagram above.

### Composio tools are fully usable with CrewAI.

This will need more exploration and thorough testing, but based on my early tinkering Composio looks very promising.

Pricing [here](#). Since they "host" the connection tokens to different services, their pricing model is around user accounts. I think this is a good solution for a fast time-to-market, and if we see that lockin becomes an issue, we can always switch their Tools for our own ones (open source ones), where we control the connection tokens.

📅 2025. Feb 26., Wed I'll work closely with Bartek to finally bring some of this together.

 Anna Barta

 2025. Feb 25., Tue

In the morning I had a longer chat with Viktor, aligning on future projects and topics. Then we had the stand-ups and monthly presentation together. After lunch we had a longer session with Christoph discussing some topics around navigation and in general.

Later in the afternoon I went into the Hermes project again. I wasn't able to focus as much as I planned (pulling all concepts together), but I created some further designs here:



**Feb 25., Tue**  
Co-editing / Cursor · Looked into Cline and Void to compare them to Cursor with my use cases bas...  
Last Edited 2/25/2025

I think I have a better understanding about some of the possibilities and I'm still excited about Cursor and what it is capable of. I feel that if on Thursday/upcoming days we will see the directions more clearly then we will be able to make further decisions about the layout, placement of buttons etc. a lot more easily (at least for an initial version then as we use it adjust). What are the bits we want to pull together in the initial version will be really exciting.

 2025. Feb 26., Wed

I'll have a meeting heavy day.

↑ Daily Notes - 2025 February

## 2025. Feb 25., Tue



**Project Hermes**  
Bartek Legięć · Craft — A fresh take on documents · Video · Added the UI for the context items · Updated the UI of t...

↑ 2025. Feb 25., Tue

## Project Hermes

 Bartek Legięć



**Craft — A fresh take on documents**  
Get started today, it's free.  
[https://staff.craft.do/?\\_appVersion=-pr-5517](https://staff.craft.do/?_appVersion=-pr-5517)



- Added the UI for the context items
- Updated the UI of the dropdown a bit
- Added the actual content generation after you fill in all of the context
- Also started passing the context to the chat so it also can use it



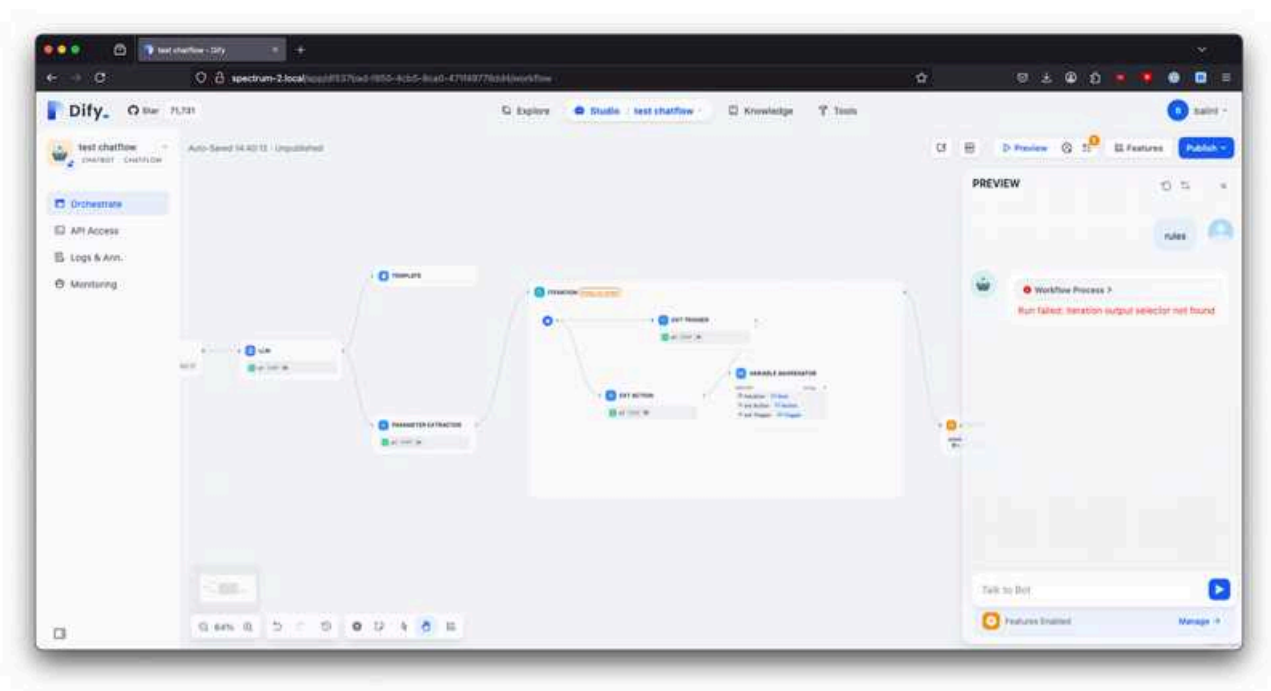
## Proposed (potential) architecture

Proceeded with different multi-agent approaches. Very rough notes [here](#).

Tried out different approaches with other frameworks in Python ([PraisonAI](#) - [serve and answer via tools](#) on MD, [parse rules.txt](#)), [CrewAI](#), and [Dify](#).

Dify is the first no-code solution I got up and running locally. Created a [simple example](#) for extracting rules and structured rule objects - some parts are still fuzzy.

Dify is great for "simple" point and click, visual flow prototyping. Its advantage is ease of use and observability during the proto phase. Its downside is it's designed to work "remote" only: all tools (or "external knowledge") are designed to be called as APIs conforming to their auth and query interface. It's not a blocker, but won't work for quickly trying new flows.



Previously I'd been looking at agent frameworks that support Typescript / JS, however, digging deeper into them the defacto standard is Python. Apart from the downsides of introducing a new language into the stack, it can be hosted in a container somewhere, and prototyping becomes very fast.

PraisonAI seems to be a dumbed-down version of CrewAI. The samples I built are okay up to a certain complexity, then don't allow for fine-grained control over agents (which we would definitely need).

CrewAI looks very promising, and after understanding how PraisonAI works I'm confident I can build prototypes rapidly with it. This is something I'd timebox tomorrow to have hands-on experience with it.

Both PraisonAI and CrewAI support different knowledge sources, however, CrewAI is miles ahead in terms of the out-of-the-box tools they provide. PraisonAI (and most frameworks) give us the standard web toolkit - search, etc. - but CrewAI has lower-level tools (JSON/MD knowledgebase from FS, etc) which enable us better.

My experience is if we want to make this a truly "chat-first" experience with deeper, more advanced logic being run in the background, a multi-agent approach is the way to go. For real-time chat applications, agent-based flows can get very slow - so confining their processing to "human-in-the-loop" scenarios when real-time is a criteria would make sense. My experiments with rules.txt were quite slow, but extracting rules can be done quickly - while heavy-lifting is done in the background.

Experiments today have raised the question of what exactly do we want to do with these agents? Currently my thinking is§

- interpret `rules.txt` and define triggers,
- perhaps run one-shot (user manually triggered) automations from `rules.txt` (question is how from a UX standpoint - simple chat?)
- run Actions from Triggers → this demands deeper context from a vector DB to get Actions right, as my experiments with just adding 1k MD files and expecting the result quickly were not successful. It would take up to 60s to process them
- add more UI-related functionality

My suggestion would be to start with `rules.txt` interpretation, extracting the exact trigger, and queueing the freeform Action in the background for processing. Currently I don't see the rules engine executing these Actions to take a formal definition of Actions, unless it's a simple text transform or a one-shot summary task. Let's discuss further.

 Anna Barta

 2025. Feb 24., Mon

In the morning I had weekly planning and alignment calls. We also had some nice discussions with Viktor about future projects and planning.



**Feb 24., Mon**  
TLDR I looked into text editing with Cursor AI as I thought it can be interesting to understand due to...  
 Last Edited 2/24/2025

 2025. Feb 25., Tue

- Continue planning with Viktor
- Workshop prep with Christoph
- Look into threading experience and continue sidebar / navigation designs.

↑ Daily Notes - 2025 February

## 2025. Feb 24., Mon

---



**Project Hermes**  
Plan for the first part of the week from Balint Orosz · For Context : Right now we are rambling in a number of differe...

↑ 2025. Feb 24., Mon

## Project Hermes

---

## Plan for the first part of the week from Balint Orosz

**For Context** : Right now we are rambling in a number of different directions, and **this is intentional**. We want to spend some time (1-2 days) in different domains to understand these a bit better. We are looking at how easy some pieces of technologies are to integrate, as well as how reliable they work / trying to look at how complex the implementations of these would be.

We can assume that there will be providers/open source solutions for each of the problems we are looking at - and as other (great) teams are working on these, eventually they will get there where these will work "out of the box". Our goal is not to create unique breakthroughs in these categories (also it would be dumb to believe we can in such a short timeframe), **but rather to find the right technologies / capabilities we can integrate into a seamless user experience**. (I.e. **package as a product**) within the next 6 weeks or so.

As a recap, we right now have the strong beliefs that :

1. **Providing Context upfront** (and likely making this context re-usable) - may this be instructions or desired outcomes of 'situational' information for the LLM - can have a massive difference in the quality of outcomes. We hope that we can create a better UX for providing context than having to enter these via chat messages and hoping the AI will understand.
2. **Having a Document/Canvas on as the main area**, and having chats / other panels as a secondary area, when well integrated enable a great content creation/refining experience.
3. **Free text based configuration** can work very well, but we likely need to add UX guardrails around them as If it's too open the chance of failure is high. (I.e. we say we define the input flow in text - make it clearer what the user can expect)
4. UX Ideas around **blending ai which writes in document and ai which you can chat with via drill-down**. (No action needed, we are still confident, actions on me)

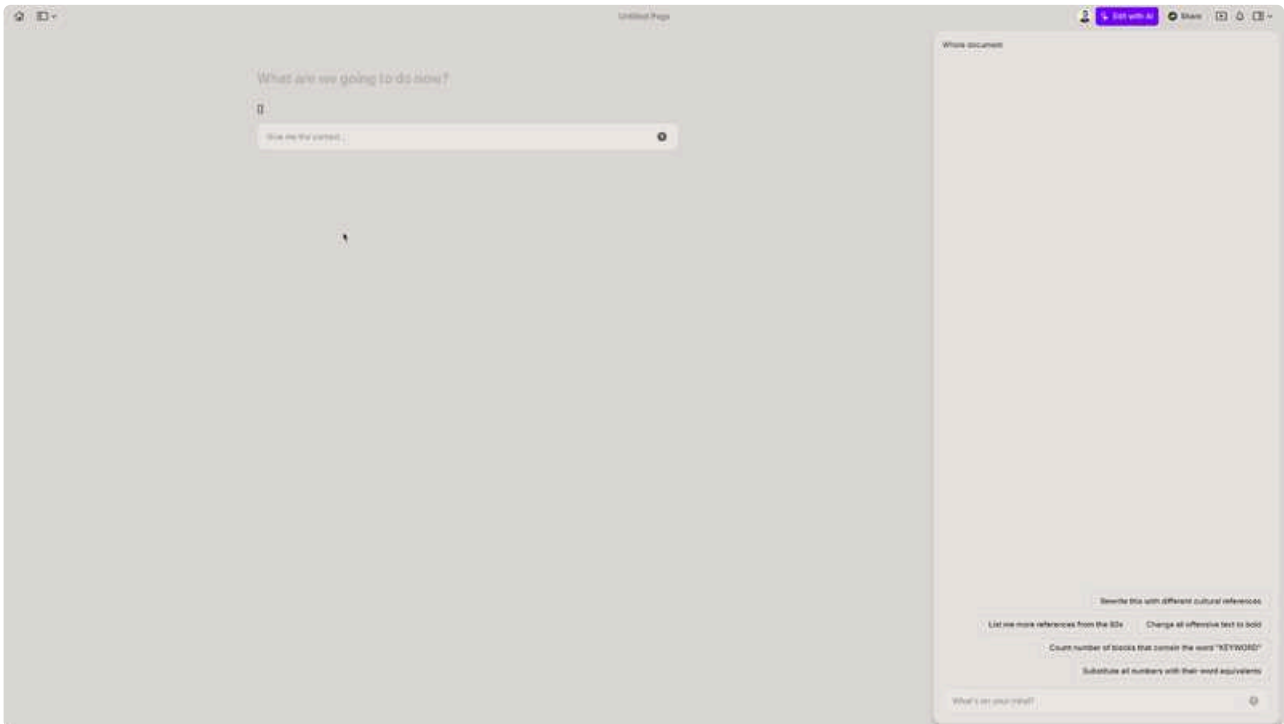
**Our goal is that on Thursday** , during our workshop **we can make some decisions** of which areas to drop, and which areas to pursue (dive deeper in), and how we can validate each.

As of such, my suggestions to work on for Mon/Tue are as follows :

- **Bartek - Gather more insights re : [1] Providing Context Upfront + [3] Free text based configuration with limitations**
  - On Monday continue with the prototype as you described. It would be great if we could eventually get this to a state where it can serve not just as a functional prototype, but also give us a "feeling" for features. For this we would need to remove all noise (Craft navigation buttons etc). It would be crucial that after we provide the context we can run a (set of) queries against it.
  - On Tuesday/Wednesday (depending on how we progress with the previous one) it would be interesting to test if we can get a reliable conversion from text → wizard and follow up with the refinements.
- **Balint - Gather more insights re : [1] Providing Context Upfront**. (Both from external sources and from existing content - i.e. previous notes)
  - On Monday continue with the multi agent model. Let's see where we get, but it might require more time for us to get this working well. (At least based on your notes/progress it seems "trivial" at the high level but non-trivial when it comes to details. Which leads me to question if we will be able to productions such system in a short timeframe.)

- On Tuesday / Wednesday I'm keen we try out some more e2e solutions, which might be more limited in possibilities, but are more robust / easier to integrate. On the Thursday workshop I want us to be informed about our short term capabilities in terms of what's possible. Part of this is to understand what open source/close source solutions exist for connecting external data / doing RAG for existing content / performing actions on data. Another part is picking the most promising ones, and doing some basic test against them. (Probably just looking at popular GitHub repos with lots of stars forks / scanning hacker news etc can help. i.e. [this kindof promises exactly what we need to test stuff](#), [this one seems quite interesting re context](#), [this one could help us to understand how to set up a simple flow](#), and I think there's at least 20-30 popular libs etc which might be worth to take a Quick Look at)
- **Anna** - Gather more insights/ideas re [2] (**Canvas main area**)
  - Looking at your calendar Monday will be full.
  - On Tuesday/Wednesday it would be great if you could do some visual explorations around making the side panel more powerful. Areas I'd be interested :
    - Changing the size / move to full size (so when the user is immersed in chat how to do it)
    - Visual style - is the floating overall a good one, or do we have some other ideas we want to visualise
    - Stacking/Navigation/History. I.e. Previous chats, how to navigate between items in the stack etc.

On Friday I have focused on asking the AI for what context do I need to fill in. I have also a basic UI for it where I can go step by step and provide the info.



I cannot use the info in any way or edit it whatsoever but we are going into the right direction.

I should be able to **finish most of this iteration today** (display the responses nicely, allow adding custom context, allow editing context answers and most importantly generating the document out of the provided context).

What will be left is integrating it with the chat on the right and persisting the answers and context about which we asked.

Friday I was experimenting with LangChain / LangGraph to build a plan-execute-replan multi-agent.

Execution is simple: based on a defined graph, plan how to accomplish a task, given the tools available. I gave it only 2 tools: `getDocument()` and `getDocumentIds()`.

With the prompt `what is the longest document?` First steps were promising in planning:

```
1 {
2   planner: {
3     plan: [
4       'Identify the documents to compare.',
5       'Determine the criteria for measuring the length of a document (number of
6       words, pages, characters, etc.).',
7       'Measure the length of each document using the chosen criteria.',
8       'Compare the lengths of all documents.',
9       'Identify the document with the greatest length as the longest document.'
10    ]
11  }
```

After a while, however, the model started hallucinating and was trying to pull documents by title:

```
1 {
2   replan: {
3     plan: [
4       'Receive specific document titles, IDs, or any identifying details from the
5       user to identify which documents will be compared.',
6       'Gather the content of each identified document to prepare for length
7       measurement.',
8       'Measure the length of each document using the chosen criteria (e.g., number
9       of words, pages, or characters).',
10      'Compare the lengths of the documents to determine which one is the
11      longest.',
12      'Inform the user about the longest document based on the chosen criteria.'
13    ]
14  }
```

It even got to the point where it wanted to ask IT Support for access to the document management system



With right prompting `which is the longest document by character count?`, it got closer:

```
1 {  
2   planner: {  
3     plan: [  
4       'List all available documents.',  
5       'Count the characters in each document.',  
6       'Compare the character counts to identify the longest document.',  
7       'Report the longest document by character count.'  
8     ]  
9   }  
10 }
```

However, no documents were actually read.

This was the first multi-agent model I built. It was a ride. And a learning experience. It's still not working as intended, I will be working on it Monday.

Code can be found in repo: <https://github.com/lukilabs/balintb-craftai/tree/main/agent/rule-chain>



 Anna Barta

## Yesterday

I had meeting in the first half of the day. Other than the stand ups we had a shorter weekly call with Peter to share knowledge, we had a session discussing Lisa's research results with her and Viktor, we also had a longer call with Viktor about planning and multiple projects.

In the second half of the day I created a few visuals for ideas we had in the previous days. I also looked more into MCP or rather intelligent agents with context.



**Feb 21., Fri**  
In the second half of the day I created a few visuals for ideas we had in the previous days. I also loo...  
 Last Edited 3/6/2025

*lllll*

## Today

- Look into Hermes sidebar experience
- Collab w Christoph on nav

↑ Daily Notes - 2025 February

## 2025. Feb 21., Fri



**Project Hermes**  
--- · Guides for the Stand ups while Balint Orosz is away : · Balint B, Bartek and Anna each talk 5-10 minutes about...

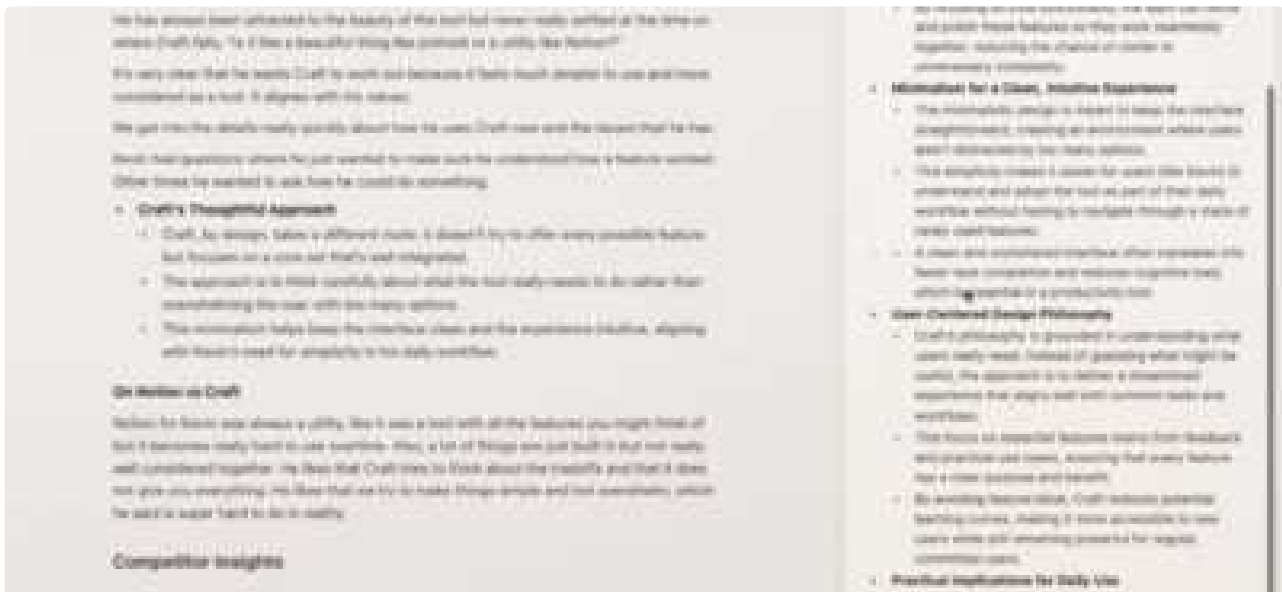
↑ 2025. Feb 21., Fri

## Project Hermes

Guides for the Stand ups while  **Balint Orosz** is away :

- Balint B, Bartek and Anna each talk 5-10 minutes about what they did, what are their challenges, and what are their thoughts. Focus on educating each other, quirks, thoughts things like that.
- In case of engineering, do a demo! Share the challenges you had, edge cases you've occurred, and potentially weird/interesting things you had to do to overcome them!

 **Balint Bartfai** to confirm with everyone (Anna & Bartek) if they are blocked, or have everything they need to work the given day. If blocked, and I need to unblock, write me.



The editing/threading/block handling for AI messages / Context providing etc works well.

Saving the chats inline isn't there yet. But not even sure we need it if we will have the context where we can add these to, we might just have it in previous chats.

Have fun playing with it - it's a wee bit buggy here and there, but manageable imo.

Here's a more "Create a report focused" use-case :



Next Steps on Proto for Bartek / Anna :

**Get Context Demo**  
We are aiming to do a prototype to understand if a workflow helps the users to provide more accur...

Last Edited 2/20/2025

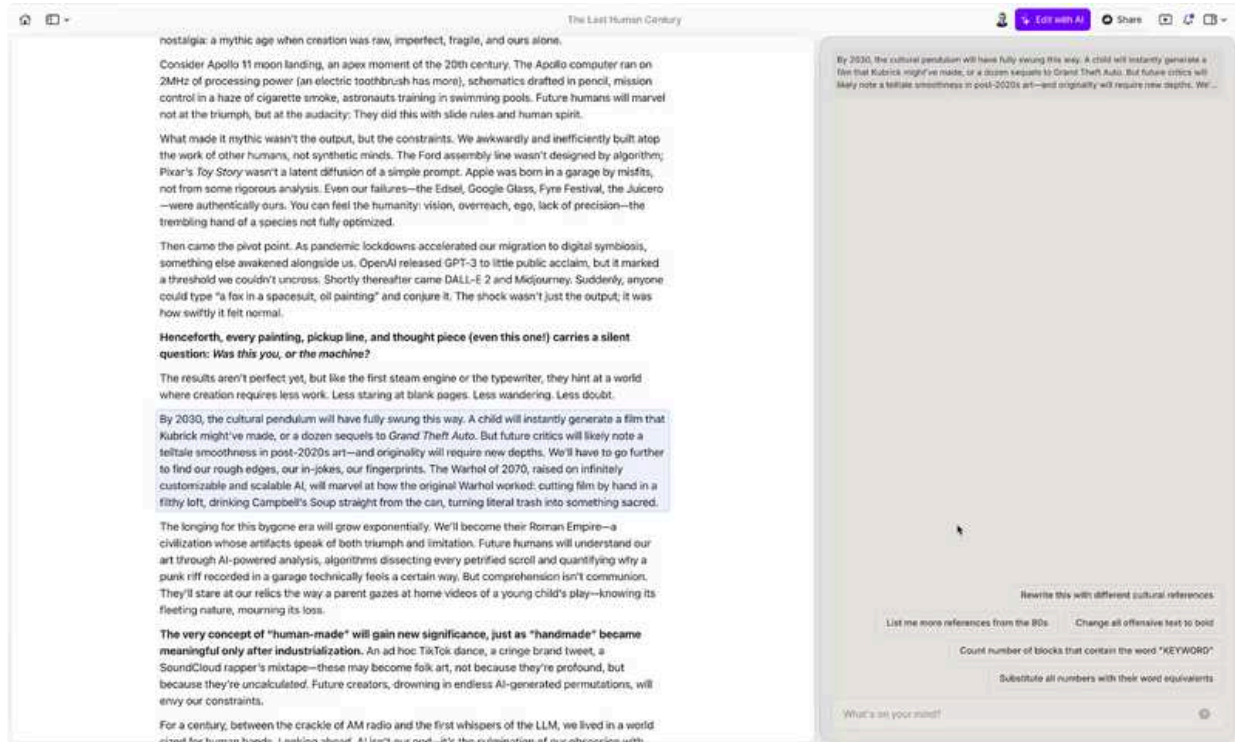
**CRAFT**

## Craft — A fresh take on documents

Get started today, it's free.

 [https://staff.craft.do/?\\_appVersion=-pr-5499](https://staff.craft.do/?_appVersion=-pr-5499)

- I've added some more pizzazz to the UI → a bit nicer chat window



Disclaimer 1: The AI has a problem with a word "rewrite" it will delete the blocks selected and create new ones instead. Keep that in mind (you can always add *Do not delete the blocks. Just replace their contents* at the end)

Disclaimer 2: It sometimes throws a strange error that says something like *Cannot access documentActions before initialisation*. If this happens to you, please send me the generated code that can be found in the console

- The AI now gets the full history of the chat, so it can respond to prompts that would require some context from previous messages.
- Added response streaming for the "information" responses.
- Created a new Vercel lambda that will return API tokens for OpenAI and Anthropic (thanks

 Balint Bartfai )

It requires authentication with a token ( `1db0cacc4732fdb2720460b1f238dbe1` ) for the first time and then it sets a cookie that lasts for a week so you have to enter the token only once (you'll get prompted for it when you enter the page)

▼ Here's the code for the service

```
1  import crypto from "crypto";
2  import { serialize, parse } from "cookie";
3
4  // Security Constants
5  const SECRET_KEY = process.env.SECRET_KEY;
6  const API_TOKEN = process.env.API_TOKEN;
7
8  // API Keys
9  const OPENAI_API_KEY = process.env.OPENAI_API_KEY;
10 const ANTHROPIC_API_KEY = process.env.ANTHROPIC_API_KEY;
11
12 if (!SECRET_KEY || !API_TOKEN || !OPENAI_API_KEY || !ANTHROPIC_API_KEY) {
13   throw new Error("Missing environment variables");
14 }
15
16 // HTTP Status Codes
17 const METHOD_NOT_ALLOWED = 405;
18 const UNAUTHORIZED = 401;
19 const OK = 200;
20
21 // Cookie Settings
22 const COOKIE_NAME = "auth_token";
23 const COOKIE_MAX_AGE = 60 * 60 * 24 * 7; // 1 week
24
25 // Headers
26 const CORS_HEADERS = {
27   "Access-Control-Allow-Credentials": "true",
28   "Access-Control-Allow-Methods": "GET, OPTIONS",
29   "Access-Control-Allow-Headers": "Authorization, Content-Type",
30 };
31
32 const AUTH_SCHEME = "Bearer";
33
34 function generateToken() {
35   const token = crypto.randomBytes(32).toString("hex");
36   const signature = crypto
37     .createHmac("sha256", SECRET_KEY)
38     .update(token)
39     .digest("hex");
40   return `${token}.${signature}`;
41 }
42
43 function verifyToken(authCookie) {
44   if (!authCookie) return false;
45   const [token, signature] = authCookie.split(".");
46   if (!token || !signature) return false;
47   const expectedSignature = crypto
48     .createHmac("sha256", SECRET_KEY)
49     .update(token)
50     .digest("hex");
51 }
```

```

52     return crypto.timingSafeEqual(
53         Buffer.from(signature),
54         Buffer.from(expectedSignature)
55     );
56 }
57
58 export default function handler(req, res) {
59     const origin = req.headers.origin;
60
61     if (origin) {
62         Object.entries(CORS_HEADERS).forEach(([key, value]) => {
63             res.setHeader(key, value);
64         });
65         res.setHeader("Access-Control-Allow-Origin", origin);
66     }
67
68     if (req.method === "OPTIONS") {
69         return res.status(OK).end();
70     }
71
72     if (req.method !== "GET") {
73         return res.status(METHOD_NOT_ALLOWED).json({ error: "Method Not
Allowed" });
74     }
75
76     let isAuthorized = false;
77
78     const cookies = parse(req.headers.cookie || "");
79     const authCookie = cookies[COOKIE_NAME];
80     const authHeader = req.headers.authorization;
81
82     if (authCookie) {
83         isAuthorized = verifyToken(authCookie);
84     } else if (authHeader) {
85         const [scheme, token] = authHeader.split(" ");
86         if (scheme === AUTH_SCHEME) {
87             isAuthorized = token === API_TOKEN;
88         }
89     }
90
91     if (!isAuthorized) {
92         res.setHeader("WWW-Authenticate", AUTH_SCHEME);
93         return res.status(UNAUTHORIZED).json({ error: "Invalid credentials" });
94     }
95
96     const token = generateToken();
97     res.setHeader(
98         "Set-Cookie",
99         serialize(COOKIE_NAME, token, {
100             httpOnly: true,
101             secure: true,
102             sameSite: "None",
103

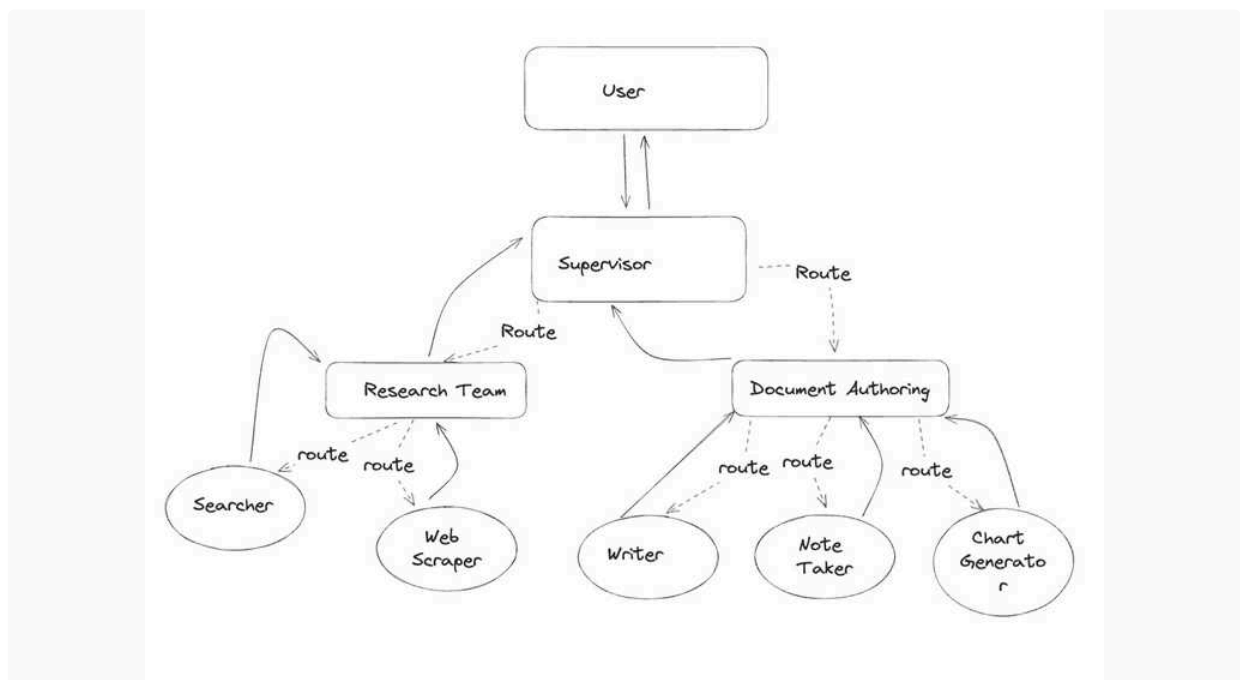
```

```
104     path: "/",
105     maxAge: COOKIE_MAX_AGE,
106   })
107   );
108
109   res.json({
110     openai: OPENAI_API_KEY,
111     anthropic: ANTHROPIC_API_KEY,
112   });
113 }
114
```

Spent a lot of research time around MCP. Specifically, understanding if MCP is the right level of abstraction we need to be reasoning about (calling APIs, etc). No!

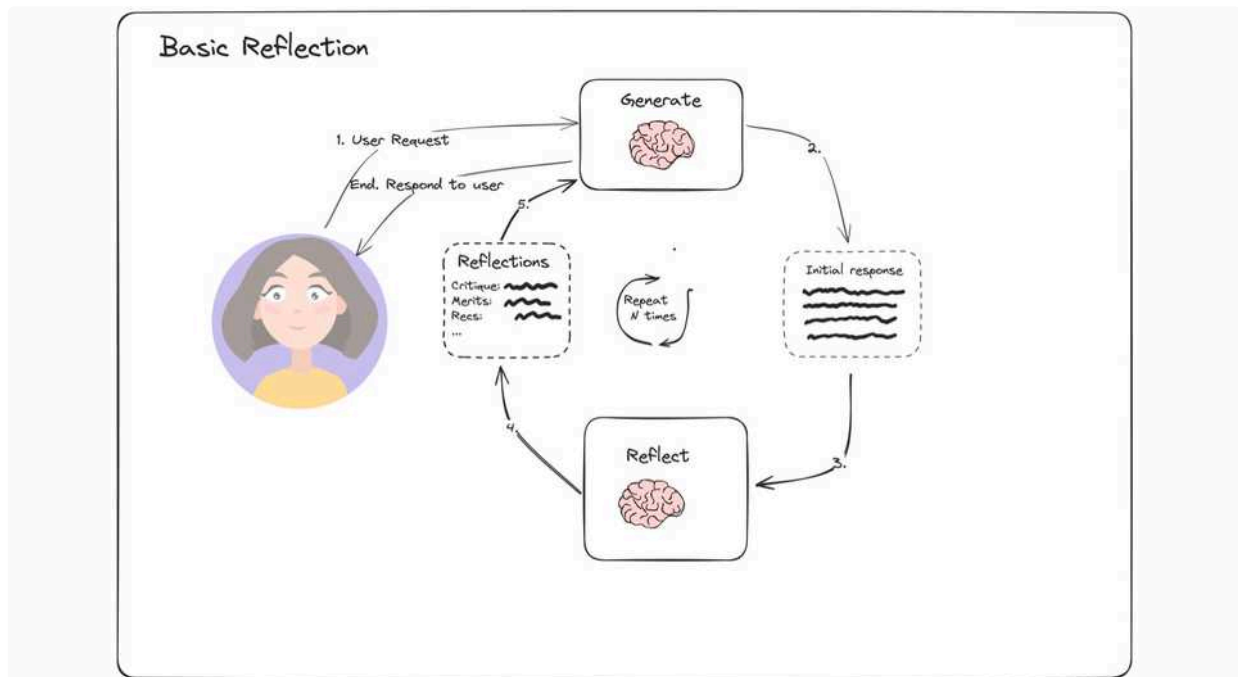
MCP is just the language any ChatLLM client can speak if they wish, which allows the **client** (not the ALM) to call tools *on behalf* of the ALM. The real magic doesn't happen there. What we are building is essentially a **hierarchical, orchestrator-based agent-team (multi-agentic) model, with human-in-the-loop reflection** capabilities, in which agents are based on - not necessarily the same model - ALMs. To break this down:

- **hierarchical:** calls to Tools, which call Tools, which... etc
- **orchestrator-based:** there's a central interface to the user (a chat thread), which is handled by a single orchestrator agent coordinating work. Not necessarily [plan-and-solve](#) at this level
- ▼ **agent-team:** the orchestrator breaks down problems (in our case, "Rules" in `rules.txt`) into smaller tasks accomplished by teams of agents. The teams then decide on execution plans on their own level, *semi-autonomously*\* from the orchestrator



- ▼ **human-in-the-loop, reflection:** after coming up with a plan, there will be multiple passes (loops) of the same "graph" being traversed with agent-agent calls. Reflection allows the MAA to check itself and explain reasoning. Human-in-the-loop allows checkpoints along that path so that we're able to modify the execution plan, retrieved entities, etc.





*\*semi-autonomously: with shared context*

Spent time reading whitepapers around multi-agentic architectures (see research doc below with links) and different frameworks that allow us to build them. So far, [LangChain](#) (and LangGraph) are top of my list for frameworks. For rapid prototyping of the topology and work orchestration we could go with a no-code, config based one like [Praison AI](#). These are things I'll try out tomorrow.

Prototyped vector embeddings and search with [Qdrant](#) ([code](#)) - this needs proper tokenising. A multi-agent system needs some form of shared context, otherwise all context would need to be encapsulated into prompts (or structured inputs). This can lead to inaccuracies and hallucination → we need to be able to embed documents and potentially Anchors (or just entities) into a vector DB.

Tomorrow my plan is to define the actual agents we would need. These should align with the initial entities I had in mind for `rules.txt` (*Action, Trigger, Transform*, etc), however, "detection" and execution would need to be separate agents.



**Feb 20., Thu**

Continuing on the MCP track. • Main learning: MCP is just the interface (just the protocol) with for...



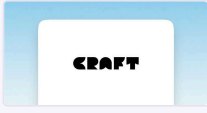
Last Edited 2/20/2025

 Anna Barta

## Yesterday

Release status, stand ups, check-in with Viktor and meeting with Balint. Next to this I worked a bit on the quarterly check-in.

Then I looked into the Hermes project and the context providing flows:



**Feb 20, Thu**  
Continued working on the idea to provide more context for the user. I spent way too much time to fi...

 Last Edited 2/20/2025

*Handwritten scribble*

## Today

Standups, Peti check-in, Viktor check-in, research alignment call. I need to work a bit more on the quarterly check-in and also do some planning and prep for next week.

Use more the new version of Balint's Hermes app, and also look at the context providing flow with a fresh pair of eyes. Look into MCP more deeply.

↑ Daily Notes - 2025 February

## 2025. Feb 20., Thu



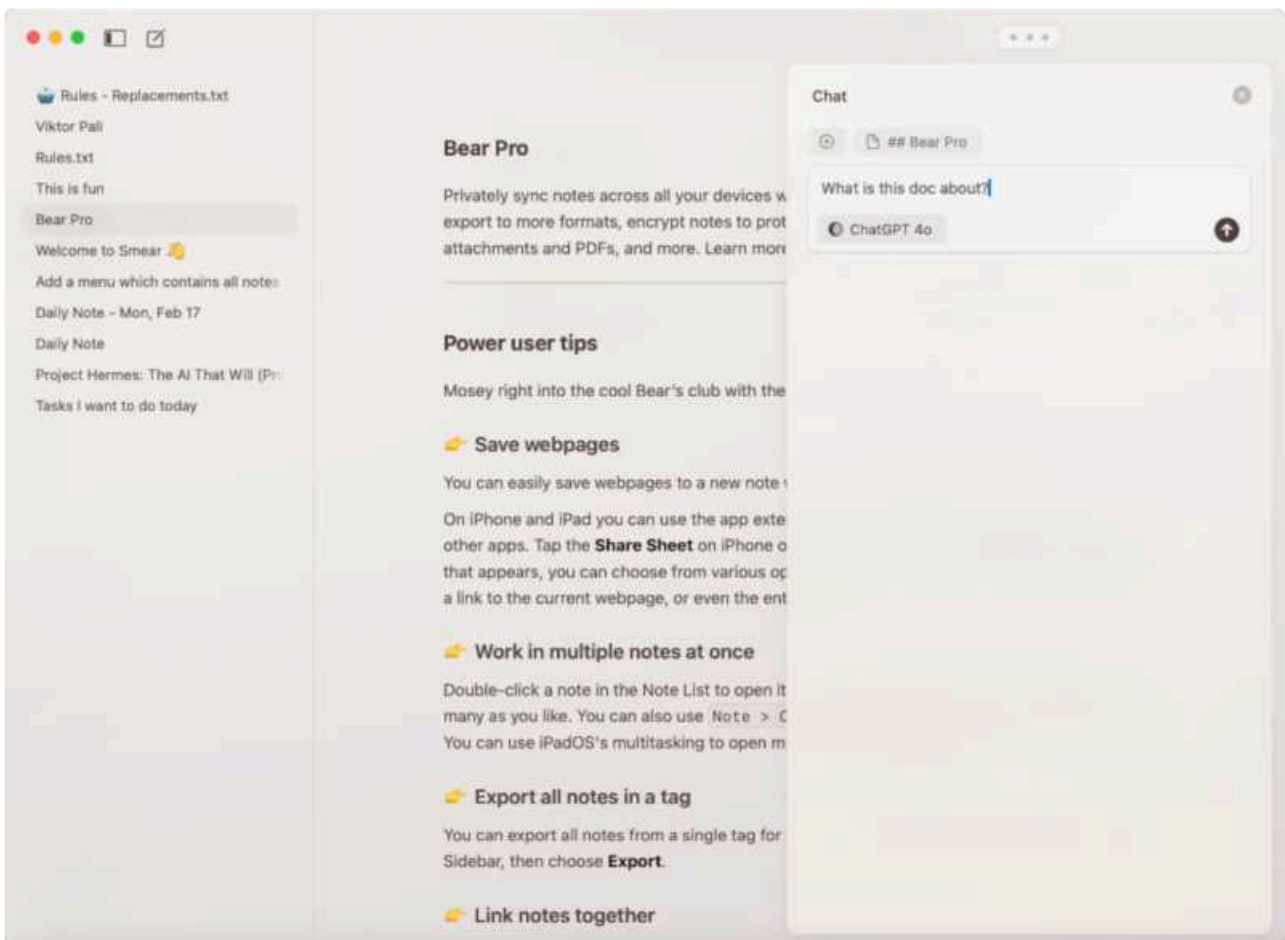
**Project Hermes**  
--- · Balint Orosz · A little bit more distractions than I planned today, but made progress with bringing in the AI chat i...

↑ 2025. Feb 20., Thu

## Project Hermes

 Balint Orosz

A little bit more distractions than I planned today, but made progress with bringing in the AI chat in the the BlockEditor Proto.



Tomorrow I'll be a lot faster, and I think I will be able to bring in the interactions from Anna into it, so she can give it a proper drive from Friday.

Also, since I can upload the entire dataset anywhere, it could be interesting if  Balint Bartfai can than use the contents here as an MCP server, so we could than just add content to the app, and also use the AI in app via MCP. This could enable testing of anchors etc as well.

 Balint Bartfai

MCP server `mdserve` is up and running on Bartek's Travel Blog corpus of 1,1k documents. Steps and learnings detailed in [research notes](#). Code is [here](#) with instructions in [README](#).

I've settled on delivering MCP servers to agents via Docker images. Tools like [Goose](#) are easily configurable to toggle MCP servers on/off, which speeds up testing significantly as opposed to building from source, running the server, etc. It's just a docker pull [ghcr.io/lukilabs/mcp/mdserve:latest](https://ghcr.io/lukilabs/mcp/mdserve:latest) and appending the config for the agent tool. See [README](#).

I think we're at a stage now to be able to integrate Bartek's Transform solution and some MCP servers to see the agent flow end-to-end. Let's catch up 📅 2025. Feb 20., Thu .

Another learning from today was **Claude Desktop is terrible for our use-case** (MD docs queried via MCP) due to the context window being too small. Nondescript error messages didn't help debugging this either. The tool I've adopted now is [Goose](#). It provides a Desktop version and a CLI. CLI can be used for rudimentary evals not just for prompts but potentially different configs of MCP servers. It allows defining essentially the same server but with different env vars and treats them as separate servers - which is easy if we can adopt the container approach outlined above. Config can be scripted, we can easily build our own test harness if need be. And it's open source Rust: I'm really interested in their model of agents - extensions (they call MCP extensions) - different LLM providers. I'm happy there's an agent project which is not Python.

I'll need to expand the capabilities of `mdserve`: specifically I'm wondering if MD docs can be defined as resources instead of tools (there's really no mutation taking place). I'm also going to spend some time on coming up with a solution to Anchors. The main question is where do we prompt? We can prompt in the MCP server and abstract away Anchor identification (which might not be the best - are we isolating context to individual docs?) or is this the agents job.

A more general question would be where are the boundaries between agent (which executes the complete flow) and MCP tools. If we mix them up by overlapping too much debugging and reasoning about the "quality" of the agent becomes very complex. Let's spend some time discussing separation of concerns here.

 Anna Barta

## Yesterday

In the first / bigger half of the day I had multiple calls. In the morning I also went into the navigation project a bit more, and we also had a longer call w Christoph to talk and collab.

Then I looked into the Hermes project. I wasn't able to come up with new strong visuals, but spent time understanding more the context bit:



**Feb 19., Wed**  
Idea from Balint · Image · Context from Balint's daily note · Note from Anna · I often start discussi...  
 Last Edited 2/19/2025

*Handwritten scribble*

## Today

- Quarterly check-in prep (as line manager and as IC)
- Meeting with Peter W, meeting with Balint
- Hermes: context

## Early Bird

**CRAFT**

### Craft — A fresh take on documents

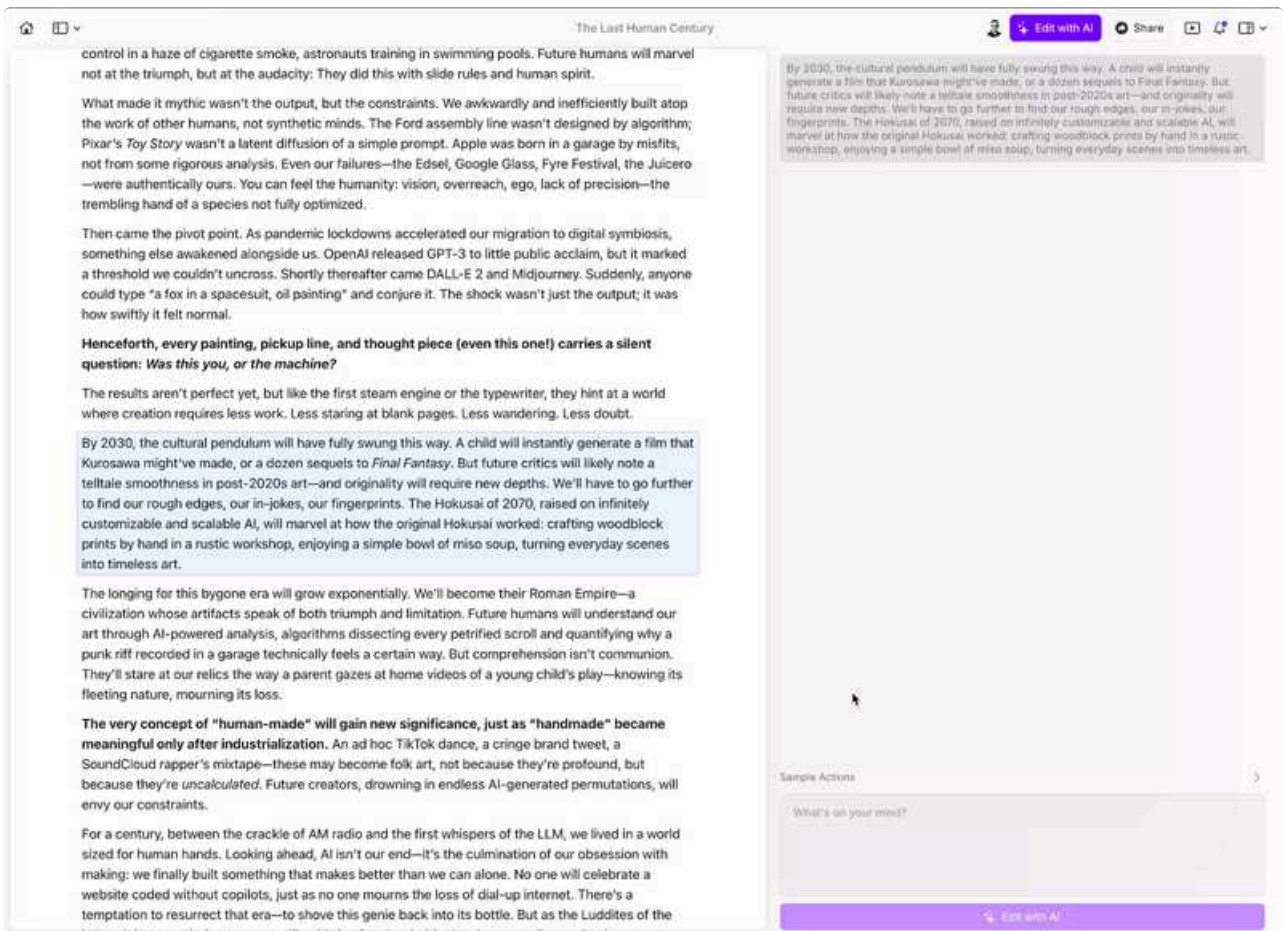
Get started today, it's free.

 [https://staff.craft.do/?\\_appVersion=-pr-5499](https://staff.craft.do/?_appVersion=-pr-5499)

Note: You'll need to provide your API keys (OpenAI and Anthropic).  
DM me if you want a quick API key for Anthropic.

Yesterday I've worked on figuring out the way to be able to "route" the logic for the what should happen.

In this example whenever the user asks for info, then I past everything to an LLM directly, otherwise I do the code generation.



The screenshot displays the Craft application interface. The main document, titled "The Last Human Century", contains several paragraphs of text. The text discusses the future of humanity, the impact of AI, and the loss of human-made artifacts. Key excerpts include:

- "control in a haze of cigarette smoke, astronauts training in swimming pools. Future humans will marvel not at the triumph, but at the audacity: They did this with slide rules and human spirit."
- "What made it mythic wasn't the output, but the constraints. We awkwardly and inefficiently built atop the work of other humans, not synthetic minds. The Ford assembly line wasn't designed by algorithm; Pixar's Toy Story wasn't a latent diffusion of a simple prompt. Apple was born in a garage by misfits, not from some rigorous analysis. Even our failures—the Edsel, Google Glass, Fyre Festival, the Juicero—were authentically ours. You can feel the humanity: vision, overreach, ego, lack of precision—the trembling hand of a species not fully optimized."
- "Then came the pivot point. As pandemic lockdowns accelerated our migration to digital symbiosis, something else awakened alongside us. OpenAI released GPT-3 to little public acclaim, but it marked a threshold we couldn't uncross. Shortly thereafter came DALL-E 2 and Midjourney. Suddenly, anyone could type 'a fox in a spacesuit, oil painting' and conjure it. The shock wasn't just the output; it was how swiftly it felt normal."
- "Henceforth, every painting, pickup line, and thought piece (even this one!) carries a silent question: Was this you, or the machine?"
- "The results aren't perfect yet, but like the first steam engine or the typewriter, they hint at a world where creation requires less work. Less staring at blank pages. Less wandering. Less doubt."
- "By 2030, the cultural pendulum will have fully swung this way. A child will instantly generate a film that Kurosawa might've made, or a dozen sequels to Final Fantasy. But future critics will likely note a telltale smoothness in post-2020s art—and originality will require new depths. We'll have to go further to find our rough edges, our in-jokes, our fingerprints. The Hokusai of 2070, raised on infinitely customizable and scalable AI, will marvel at how the original Hokusai worked: crafting woodblock prints by hand in a rustic workshop, enjoying a simple bowl of miso soup, turning everyday scenes into timeless art."
- "The longing for this bygone era will grow exponentially. We'll become their Roman Empire—a civilization whose artifacts speak of both triumph and limitation. Future humans will understand our art through AI-powered analysis, algorithms dissecting every petrified scroll and quantifying why a punk riff recorded in a garage technically feels a certain way. But comprehension isn't communion. They'll stare at our relics the way a parent gazes at home videos of a young child's play—knowing its fleeting nature, mourning its loss."
- "The very concept of 'human-made' will gain new significance, just as 'handmade' became meaningful only after industrialization. An ad hoc TikTok dance, a cringe brand tweet, a SoundCloud rapper's mixtape—these may become folk art, not because they're profound, but because they're uncalculated. Future creators, drowning in endless AI-generated permutations, will envy our constraints."
- "For a century, between the crackle of AM radio and the first whispers of the LLM, we lived in a world sized for human hands. Looking ahead, AI isn't our end—it's the culmination of our obsession with making: we finally built something that makes better than we can alone. No one will celebrate a website coded without copilots, just as no one mourns the loss of dial-up internet. There's a temptation to resurrect that era—to shove this genie back into its bottle. But as the Luddites of the

On the right side of the interface, there is a sidebar with a section titled "Sample Actions" containing a text input field labeled "What's on your mind?". At the bottom of the sidebar, there is a button labeled "Edit with AI".

Didn't figure out **yet** what to do if they ask for both? But it shouldn't be too difficult.



## Project Hermes

--- · To Discuss : · We are learning (amazing!!) and we need to adapt our activities and explorations based on what...

↑ 2025. Feb 19., Wed

## Project Hermes

---

## To Discuss :

We are learning (amazing!!) and we need to adapt our activities and explorations based on what we've learned.

Some of the important things we're aligning on/learnt :

1. As per Bartek's and Balint O's work **AI writing executable code is still hacky**. To achieve good results **we will need to be specific**. This means an "anything assistant" today is not easily possible (we need to probably create 100 different assistants and then have a router decide which one to use), but might get a lot easier within 3-6 months as models follow directions more consistently.
2. As per Anna's work, the main entity/artefact for us will be the document/note. This is how we differ from ChatGPT and other tools where the chat is the main artefact. **This is logical, as we are a writing app**. We want to build a writing app for this new world.
3. **Context makes AI useful**. Tools that have lots of context become quite useful. We can improve the output/first shot generation of models by giving them more context. But we need to ask for context, and make it easy. It's quite clear that Perplexity, Cursor, Lex are all working on improving their responses via letting the users have more control over context (and also allowing users to save context which they can re-use!)

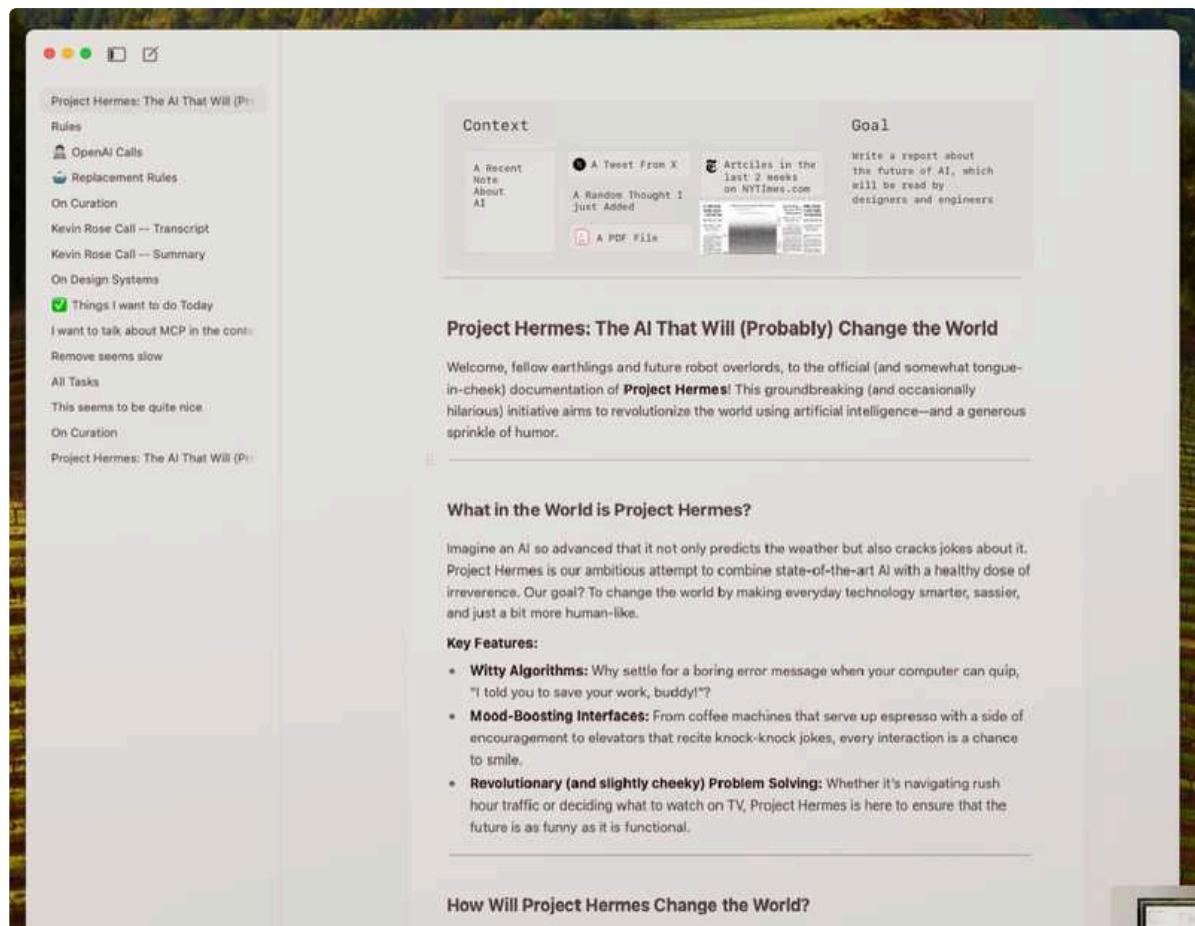
Note, I also believe that there's a point where a 90% accurate answer is 10x less valuable than a 99% accurate one - so anything we can do increase accuracy for the first shot generation is a massive value add



All of these result in :

- Let's drop for now advanced free-form transformations. **We should focus on transformations seen on Anna's prototype** (I have a selection and explicitly ask for an update). Here technically the only thing we need is the model to decide if this is a chat or a change request, and pass return values accordingly. (But as we can assume these will be smaller bits, for now full text generation by LLM is possible, so NO JS Execution for now is needed). **We will take the tradeoffs that the longer the text, the less accurate/fast it is**, but our UX allows for fine-grained refinements so it should be OK.
- **Let's explore more on getting the user to the point where they will want to use the amazing AI Editing**. This means post the first generation step, so :
  - Let's explore on helping the user set up the context (Is it a set of questions? Is it just drag and drop?)
  - Let's explore on the tech side how we can easily get context into :
    - MCP (Balint B already on it)
    - MCP from GMail, Slack, Linear etc - could we get these easily in the app? See : <https://github.com/modelcontextprotocol/servers>
  - Design side - if context is important, and our artefact is the doc, should we always add context at the top of the doc?





## What about code generated transforms / settings?

Let's get back to that later. I still like that idea, but we need something simpler - maybe the context acquiring flow for certain use-cases will be the number configuration, which can easily be done via text.

Today was mostly a day of dabbling into things, I didn't really go into anything deeply, but rather I played around with a bunch of stuff quickly

- Did some brainstorming about detecting anchors in the space. Here are my chats with DeepSeek to figure out if we can



#### Building a Context-Aware Document Search Engine with OpenAI and ChromaDB

Bartek: 1. I have a large amount of documents in markdown · Using OpenAI and ChromaDB (if you need anythin...



#### Finding and Organizing Anchor Entities in Large Text Datasets

Bartek: I have a large, text dataset. I want to somehow find "anchors" in it. · Anchors meaning people, objects, p...

- I've scraped a whole, longrunning travel blog (1k+ posts). This seems like a thing that has a bunch of connections between trips, and is talking about different topic so could be a nice testing dataset.



#### blog-posts.zip

ZIP File  
7,2 MB



#### Adventurous Kate - The Solo Female Travel Blog

My name is Kate McCulley and I travel the world for a living. My goal is to show YOU how you can travel th...

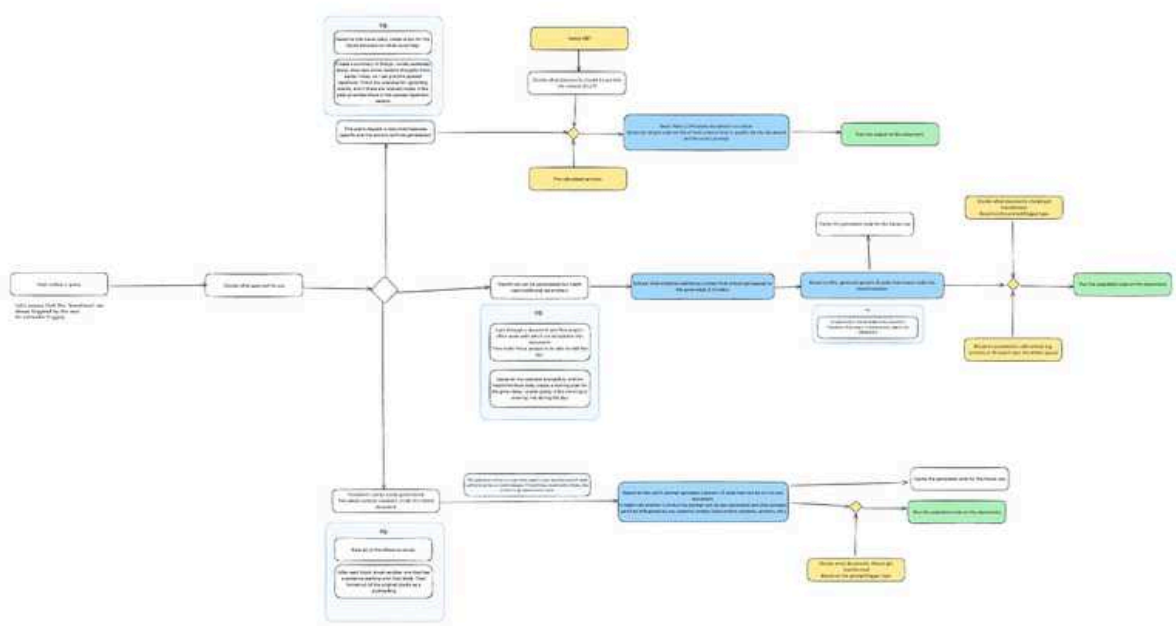
<https://www.adventurouskate.com/blog/>

- Also during the morning chat with Balint I've realised that I haven't really shared the prompt that I've used for my proto. And it is quite long and deep so it might be interesting to see.

My prompt (7k+ characters)

Craft Docs Assistant · You are an assistant built into Craft Docs. · Craft Docs is a multi-platform document editing app...

- Then I've also started mapping out the flow that I imagine would work for creating the transformers.



Excalidraw — Collaborative whiteboarding made easy

Spent time with Bartek this morning and agreed Transformations are better cached and more suited to our use-case than my AST ones.

To parallelize our work, Bartek focused on the "end" of the Transform pipeline, I started from the beginning.

General thinking around transforms. They fall into the following categories:

- "diff-style" text-only (expansion, summarisation, formatting, etc). This can be across a Space or just a Document → essentially what Bartek has been implementing
- Entity (~Anchor) transforms. Manipulating entities. Precursor/input to diff-style transforms
- (Purely) Generative transforms. We can bypass the filtering and agent-style transformation phase and generate text with only the prompt and minimal context

The beginning of the pipeline means feeding distilled context into the Transform phase. Anchors become the foundations of understanding documents, so I explored how abstract "Anchors" can be extracted from a corpus. Usage of these Anchors is still fuzzy.

▼ Example prompt for extraction

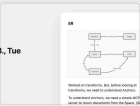
You will be provided my markdown documents. Individual documents are separated by `<!-- DOC[document name] -->`.

An entity is something that exists as itself in my written documents. It does not need to be of material existence. Abstractions and legal fictions are regarded as entities. There is also no presumption that an entity is animate, or present. An entity does not need to be just a document. Entities with the same name are separated according to context. Context is an abstract classification of said entity, which can be temporal, spatial, hierarchical, topical or literal. An "Anchor" is an abstract entity with its context inferred from the contents of markdown documents I have written.

Identify Anchors in these documents. Find as many as there are that are important to this corpus of documents. Return them in the form of: (...)

Wrote a simple MCP server to be able to read MD docs. All code in [this repo](#).

Unstructures research notes:



**Feb 18., Tue**

ER — Worked on transforms. But, before looking at transforms, we need to understand Anchors. — To un...

<https://docs.craft.do/editor/d/a74950fb-ace8-80e7-ec4d-383742d9cc03/4B5AF8EF-854F-4941-9B08-885D30AC4761?s=X7GFkndifsbc...>

 Anna Barta



### Design Team

Topics to discuss · Please add any additional topics, or topics you'd highlight from your daily note....



Last Edited 2/19/2025

## Maybe it's about Context?

Ok, this is part of the process, but I think we have an (idea) for an entry point. I had an interesting chat with Roberto, one of our investors. He was like, nowadays me writing is :

- Make this email crisper
- Summarize this meeting
- Ask questions about a doc
- ...

And he was where do I do this? Where do I write?

And how would an app like this look like?

Start with a question, what do you want to do :

- Write an email
- Research a Topic
- ...

User Chooses write an email. App asks, do you have any context for this?

- Either a screenshot of the previous conversation
- Any articles, sources you have etc..?

User pastes the conversation history. App asks : Do you have any specific intention or should I just go ahead?

- [Enter intention]
- Compose a response

AI Writes the response. We are now in edit mode, can use all sorts of in-doc AI editing, chat etc.

At the bottom there are buttons :

[Copy to Gmail], [Copy to Apple Mail]

So maybe the "customization" is the workflow of how to get context. Maybe the "plugins" are written descriptions of what to ask for a certain workflow.

Maybe that's why AI is so good in coding, as :

- Frameworks create shared context
- A codebase creates very strong context (So Cursor can rock!)

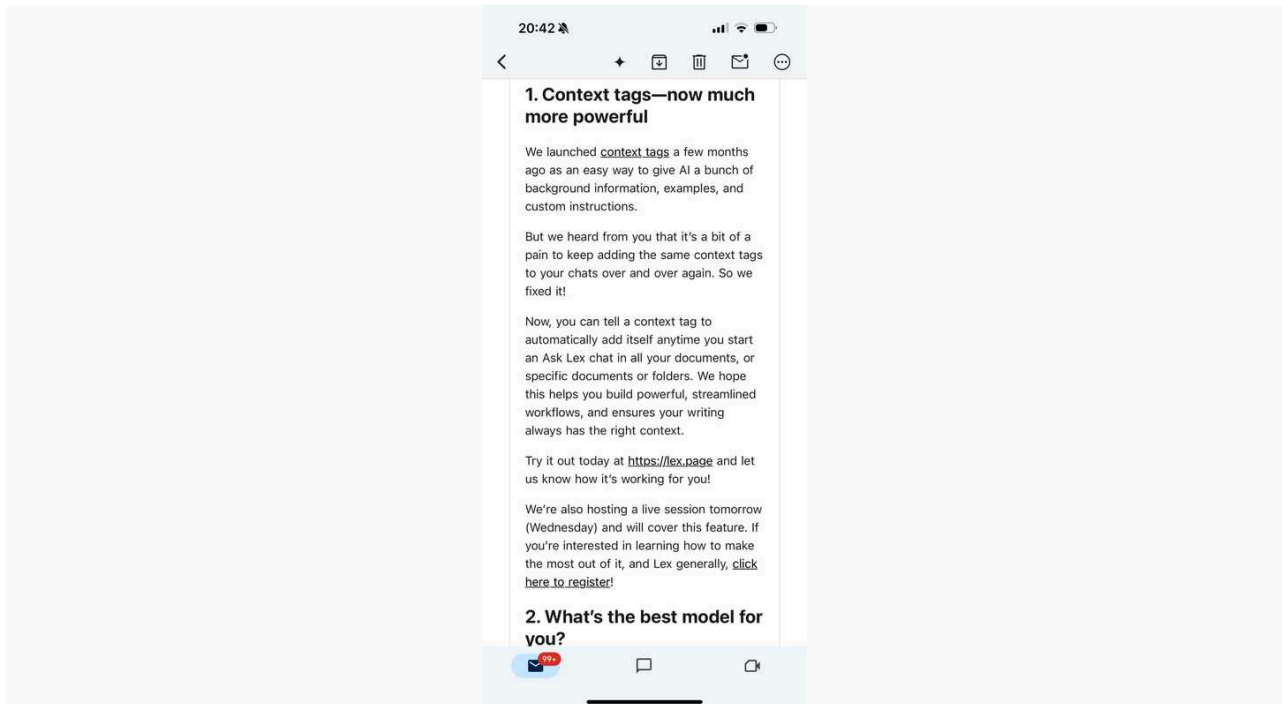
Also, maybe that's why AI meeting notes are so popular, because the transcript creates so good context.

Maybe, to be better than ChatGPT for these use cases we need better context, and **shine on areas where context is needed**. And of course the editing after is a *lot* easier and better.

But AI is about fast start, instant value. So maybe **getting the right context at the start is what is really needed for a great workflow.**



On that note Lex just improved context tags :



↑ 2025. Feb 19., Wed · Project Hermes

## Building a Context-Aware Document Search Engine with OpenAI and ChromaDB

Bartek: 1. I have a large amount of documents in markdown

2. Using OpenAI and ChromaDB (if you need anything else, please suggest)
3. Using TypeScript
4. I want to be able to ask GPT about some data
5. The whole app should be aware of "anchors" (which are people, places, etc.) that can be a thing that they can ask for
6. Note that it should be able to distinguish and have context about distinct anchors (eg. Victor my boss, is different from Viktor: my part friend, and different from Viktor as a letter in ICAO)

How do I approach this?



Raycast AI:

Here's a structured approach to implement your knowledge retrieval system:

## 1. Document Processing Pipeline

```
1 graph TD
2   A[Markdown Documents] --> B[Chunking]
3   B --> C[Anchor Extraction]
4   C --> D[Metadata Enrichment]
5   D --> E[Embedding Generation]
6   E --> F[ChromaDB Storage]
```

### Key components:

- **Chunking:** Split documents into logical sections (512-1024 tokens)
- **Anchor Extraction:**

```
1 interface Anchor {
2   id: string;
3   name: string;
4   type: 'person' | 'place' | 'concept';
5   context: string;
6   aliases: string[];
7   source_doc: string;
8 }
```

- Use OpenAI's NER (Named Entity Recognition) with custom prompt:

```
1 Identify and disambiguate entities in this text.
2 For each entity, provide:
3 - Canonical name
4 - Type (person/place/etc)
5 - Unique context identifier
6 - Alternative spellings/forms
```

## 2. ChromaDB Schema Design

```
1 const collection = await client.createCollection({
2   name: "document_anchors",
3   metadata: {
4     // Context-aware indexing
5     "Viktor-boss": "Work context: Manager at XYZ Corp",
6     "Viktor-friend": "Personal context: Childhood friend",
7     "Viktor-ICA0": "Technical context: Phonetic alphabet"
8   },
9   embeddingFunction: OpenAIEmbeddingFunction({
10     openai_api_key: "...",
11     model_name: "text-embedding-3-large"
12   })
13 });
```

## 3. Query Resolution Flow

## 1. Input Parsing:

```
1  const analyzeQuery = async (query: string) => {
2    return openai.chat.completions.create({
3      model: "gpt-4-turbo",
4      messages: [{
5        role: "user",
6        content: `Disambiguate entities in: "${query}".
7                  Return JSON with {entities: {name, type, contextHint}}`
8      }]
9    });
10  };
```

## 2. Contextual Search:

- Use extracted entities to filter ChromaDB metadata
- Combine with semantic search using query embeddings

## 4. Response Generation

```
1  const buildPrompt = (contexts: string[], query: string) => `
2  You're an expert analyst with access to these verified contexts:
3  ${contexts.map((c, i) => `[Source ${i + 1}] ${c}`).join('\n')}
4
5  Answer this query: ${query}
6  Consider these special anchors:
7  - Differentiate between similar names using context
8  - Verify facts against source material
9  - Cite sources when possible
10 `;
11
12 const response = await openai.chat.completions.create({
13   model: "gpt-4-turbo",
14   messages: [{ role: "user", content: finalPrompt }]
15 });
```

## 5. Enhancement Considerations

### • Entity Resolution Layer:

```
1  const resolveAmbiguity = (entity: string, context: string) => {
2    // Use cosine similarity between context embedding
3    // and known anchor contexts
4  };
```

- **Cache System:** Store frequent query-answer pairs
- **Feedback Loop:** Implement user correction mechanism



```

1 interface Feedback {
2     query: string;
3     expectedAnchor: string;
4     receivedAnchor: string;
5     correction: string;
6 }

```

## Tech Stack Recommendations

```

1 |— Core
2 |   |— LangChain.js - Orchestration
3 |   |— ChromaDB - Vector Store
4 |   |— OpenAI API - LLM
5 |
6 |— Infrastructure
7 |   |— Redis - Cache
8 |   |— PostgreSQL - Anchor Metadata
9 |
10 |— Monitoring
11 |   |— Prometheus - Metrics
12 |   |— Winston - Logging

```

## Cost Optimization

- Use `text-embedding-3-small` ( $\frac{1}{50}$  the cost of large model)
- Implement document versioning to avoid re-indexing
- Batch processing of documents during off-peak hours

Would you like me to elaborate on any specific component or provide sample code for a particular section?

↑ 2025. Feb 19., Wed · Project Hermes

## Finding and Organizing Anchor Entities in Large Text Datasets

Bartek: I have a large, text dataset. I want to somehow find "anchors" in it.

Anchors meaning people, objects, places and anything else that a person would ask to "anchor" around and ask more questions.

This info would most probably be used for other LLM calls.

Is vector DB something to look into? Maybe you have a better idea for a solution?

Raycast AI:

Here's a structured approach to finding "anchors" in your text dataset:

## 1. Entity Recognition Pipeline

- **NER (Named Entity Recognition):**
  - Use models like `spaCy`, `Stanford NER`, or `BERT-based models` to extract:
    - People (\*Jan Kowalski\*)
    - Organizations (\*Warszawski Uniwersytet\*)
    - Locations (\*Kraków\*)
    - Dates (\*18 lutego 2025\*)
- **Noun Phrase Extraction:**
  - Identify concrete objects/contexts using POS tagging:
    - "czerwony samochód" → \*samochód\*
    - "zabytkowy most" → \*most\*

## 2. Vector DB Strategy

- **Embedding Generation:**
  - Use multilingual models like `sentence-transformers/all-mpnet-base-v2` to create embeddings
  - Store in vector DBs like:
    - **Pinecone** (managed service)
    - **FAISS** (local/OSS)
    - **Qdrant** (geo-aware queries)
- **Query Pattern:**

```
1 results = vector_db.query(  
2     vector=embed("historyczne budynki"),  
3     top_k=50,  
4     filter={"category": "location"}  
5 )
```

## 3. Contextual Enrichment

- **Temporal Anchoring:**
  - Extract and normalize dates to 2025-02-18T10:42:17+01:00 (Warsaw time)
  - Link events to local holidays (\*Wielkanoc 2025\*)
- **Unit Conversion:**
  - Automatically convert measurements:
    - "5 feet" → \*1,52 m\*
    - "70°F" → \*21,1°C\*

## 4. LLM-Assisted Filtering

- **Prompt Template:**

```
1 Identify anchor points in this text for follow-up queries.
2 Prioritize entities relevant to Polish context.
3 Text: {text}
```

- **Post-Processing:**

- Deduplicate using phonetic algorithms (Warsaw → Warszawa)
- Add PL-specific context (identify "OSP" as fire department volunteers)

## 5. Hybrid Storage

| Method | Use Case | Example Output |

|-----|-----|-----|

| Vector DB | Semantic similarity | Finds "zamek" matching both castle and lock |

| RDBMS | Structured querying | All people born after 2000-01-01 |

| Graph DB | Relationship mapping | Show connections between politicians and companies |

## 6. Optimization for PL Context

- Handle Polish declensions (Warszawa → Warszawie → Warszawy)
- Recognize local institutions (\*PKP\*, \*Orlen\*)
- Prioritize metric units (show distances in meters, weights in kg)

### Implementation Path:

1. Start with spaCy (Polish model) for basic entity extraction
2. Add sentence transformers for semantic similarity
3. Use Qdrant for vector storage (supports EU-based hosting)
4. Implement unit-aware normalization (convert all measurements to metric)

This approach balances linguistic specificity (Polish context) with scalable architecture for LLM integration.



Bartek: How do I quickly spin up a prototype of BERT + VectorDB? I'm a JS developer so this is what I know the best



Raycast AI:

Here's a streamlined JavaScript-focused prototype using tools you already know:

### 1. Setup with Node.js Packages

```
1 npm install @xenova/transformers chromadb dotenv
```

## 2. Simplified BERT Embeddings (using Transformers.js)

```
1  const { pipeline } = require('@xenova/transformers');
2  const { ChromaClient } = require('chromadb');
3
4  // Use multilingual model for PL/EN context
5  const embedder = await pipeline('feature-extraction', 'Xenova/bert-base-
   multilingual-cased');
6
7  async function getEmbedding(text) {
8    const output = await embedder(text, { pooling: 'mean' });
9    return Array.from(output.data);
10 }
```

## 3. ChromaDB Setup (Lightweight VectorDB)

```
1  const client = new ChromaClient();
2  const collection = await client.createCollection({
3    name: "text-anchors",
4    metadata: { "hnsw:space": "cosine" } // Optimize for semantic similarity
5  });
6
7  // Add documents with metadata
8  await collection.add({
9    ids: ["doc1"],
10   embeddings: [await getEmbedding("Warszawa jest stolicą Polski)],
11   metadatas: [{ category: "location", lang: "pl" }],
12   documents: ["Warszawa jest stolicą Polski"]
13 });
```

## 4. Querying for Anchors

```
1  async function findSimilar(text) {
2    const results = await collection.query({
3      queryEmbeddings: [await getEmbedding(text)],
4      nResults: 5,
5      where: { category: "location" } // Filter by anchor type
6    });
7
8    return results.documents[0].map((doc, i) => ({
9      text: doc,
10     score: results.distances[0][i]
11   }));
12 }
13
14 // Usage for LLM context
15 const anchors = await findSimilar("Główne miasto w Polsce");
16 // Returns [{text: "Warszawa...", score: 0.23}]
```

## Prototyping Tips:

1. **Prebuilt Models:** Use `Xenova/` prefix models for auto-converted ONNX versions
2. **Local First:** ChromaDB runs in-memory without servers
3. **Polish Text Handling:** The multilingual BERT handles PL diacritics well
4. **Performance:** Batch process embeddings with `Promise.all()`

### Alternative Cloud Path (if GPU needed):

```

1 // Use Hugging Face Inference API
2 const HF_API_KEY = process.env.HF_KEY;
3
4 async function hfEmbed(text) {
5   const response = await fetch("https://api-inference.huggingface.co/pipeline/
   feature-extraction/sentence-transformers/all-mpnet-base-v2", {
6     headers: { Authorization: `Bearer ${HF_API_KEY}` },
7     method: "POST",
8     body: JSON.stringify({ inputs: text })
9   });
10  return response.json();
11 }

```

### Recommended Stack:

| Layer         | JS Tools                       | Polish Context Notes                          |
|---------------|--------------------------------|-----------------------------------------------|
| Embeddings    | Transformers.js / HF Inference | Use <code>bert-base-multilingual-cased</code> |
| Vector DB     | ChromaJS / Qdrant Cloud        | Host in EU regions                            |
| Normalization | Natural.js (POS tagging)       | Add custom PL entity rules                    |
| Deployment    | Vercel Edge Functions          | Low-latency for EU users                      |

For initial testing, try this Polish news dataset example:

```

1 const sampleAnchors = [
2   "Prezydent Andrzej Duda spotkał się w Warszawie",
3   "PKP Intercity opóźnienia wynoszą 45 minut",
4   "Wawel wprowadza nowe opłaty za wstęp"
5 ];
6
7 await collection.add({
8   ids: sampleAnchors.map((_, i) => `doc${i+1}`),
9   embeddings: await Promise.all(sampleAnchors.map(getEmbedding)),
10  documents: sampleAnchors
11 });

```

Query with `findSimilar("kto rządzi w Polsce?")` should surface the presidential anchor.



Bartek: Please don't consider that I live in PL for the future questions



Raycast AI:

Noted - I'll drop the Poland-specific context moving forward. Here's a revised JS-first prototype approach:

## Minimal Viable Prototype Stack

1. **Embeddings:** `@xenova/transformers` (BERT variants)
2. **VectorDB:** `pinecone-client` (serverless)
3. **NER:** `compromise` + custom rules

```
1 // Core Setup
2 import { Pinecone } from '@pinecone-database/pinecone';
3 import { pipeline } from '@xenova/transformers';
4
5 // 1. Choose General-Purpose Model
6 const embed = await pipeline('embeddings', 'Xenova/all-MiniLM-L6-v2');
7
8 // 2. Pinecone Setup (Free Tier)
9 const pinecone = new Pinecone({ apiKey: process.env.PINECONE_KEY });
10 const index = pinecone.index('anchor-prototype');
11
12 // 3. Entity Extraction
13 function extractAnchors(text) {
14   const doc = nlp(text);
15   return {
16     people: doc.people().out('array'),
17     places: doc.places().out('array'),
18     orgs: doc.organizations().out('array'),
19     // Custom pattern for objects
20     objects: doc.match('#Noun+ of? the? #Noun+').out('array')
21   };
22 }
23
24 // 4. Embed & Index
25 async function indexDocument(text) {
26   const { people, places, orgs, objects } = extractAnchors(text);
27   const anchors = [...people, ...places, ...orgs, ...objects];
28
29   const vectors = await Promise.all(
30     anchors.map(async anchor => ({
31       id: crypto.randomUUID(),
32       values: await embed(anchor),
33       metadata: { type: detectType(anchor), source: text.slice(0,50) }
34     }))
35   );
36
37   await index.upsert(vectors);
38 }
39
40
```

```

41 // 5. Query (for LLM context)
42 async function findRelevantAnchors(query) {
43   const queryEmbedding = await embed(query);
44   return index.query({ vector: queryEmbedding, topK: 5 });
45 }

```

## Quickstart Test

```

1 PINECONE_KEY="your-key" node prototype.js

```

## Sample Workflow

```

1 // Index sample text
2 await indexDocument(`Apple Inc. announced new iPhone models in Cupertino.
3 CEO Tim Cook revealed the A17 chip at California Theater.`);
4
5 // Query later
6 const results = await findRelevantAnchors("tech leadership");
7 // Returns Tim Cook, CEO, Apple Inc., Cupertino, California Theater

```

## Why This Stack?

- **Pinecone:** Free tier handles ~100k vectors
- **Transformers.js:** Browser/Node compatible
- **No Polish Artifacts:** Using generic entities and US-centric examples
- **Cost:** ~\$0 until scale

Need me to adjust any part for global/neutral context?

~~~~~

Bartek: if using chroma, do I need BERT? Also can you give me just one script?

~~~~~

Raycast AI:

Here's a self-contained script using Chroma's built-in embedding function (no separate BERT needed) for a quick test:

```

1 // anchor-prototype.js
2 import { ChromaClient } from 'chromadb'
3
4 const client = new ChromaClient()
5 const collection = await client.createCollection({
6   name: 'anchors',
7   embeddingFunction: {
8     // Chroma's default embedding (all-* models)
9     generate: async (texts) => {
10       const response = await fetch('https://api.chroma.dev/embeddings', {
11

```

```

12     method: 'POST',
13     headers: { 'Content-Type': 'application/json' },
14     body: JSON.stringify({ texts })
15   })
16   return response.json()
17 }
18 }
19 })
20
21 // Add sample data
22 await collection.add({
23   ids: ['1', '2'],
24   documents: [
25     'CEO Mark Zuckerberg announced Meta AI in Menlo Park',
26     'Tesla recalls Cybertruck over windshield wiper issue'
27   ],
28   metadatas: [
29     { type: 'executive' },
30     { type: 'product' }
31   ]
32 })
33
34 // Query
35 const results = await collection.query({
36   queryTexts: ['tech leadership changes'],
37   nResults: 2
38 })
39
40 console.log('Top anchors:', results.documents[0])

```

Run it with:

```

1 npm install chromadb
2 node anchor-prototype.js

```

### Key advantages:

1. No local ML models - uses Chroma's API embeddings
2. Single file with end-to-end flow
3. Built-in metadata filtering ( `where: { type: 'product' }` )

### Output would look like:

```

1 Top anchors: [
2   "CEO Mark Zuckerberg announced Meta AI in Menlo Park",
3   "Tesla recalls Cybertruck over windshield wiper issue"
4 ]

```

For production, replace the embedding function with local models ( `@xenova/transformers` ) or OpenAI, but this gives you instant anchors without ML setup.





Bartek: I want to use BERT actually



Raycast AI:

Here's a complete script using BERT embeddings with Chroma in a single file:

```
1 // anchors.js
2 import { ChromaClient } from 'chromadb';
3 import { pipeline } from '@xenova/transformers';
4
5 // 1. Initialize BERT embedder
6 const embedder = await pipeline('feature-extraction', 'Xenova/bert-base-uncased');
7
8 // 2. Create Chroma collection with custom BERT embedding
9 const client = new ChromaClient();
10 const collection = await client.createCollection({
11   name: "bert-anchors",
12   embeddingFunction: {
13     generate: async (texts) => {
14       const outputs = await Promise.all(
15         texts.map(text => embedder(text, { pooling: 'mean' }))
16       );
17       return outputs.map(output => Array.from(output.data));
18     }
19   }
20 });
21
22 // 3. Add sample documents
23 await collection.add({
24   ids: ['doc1', 'doc2'],
25   documents: [
26     'Elon Musk announced Tesla Model Y price cut in Austin, Texas',
27     'Apple unveils new M3 chip for MacBook Pro at Cupertino event'
28   ],
29   metadatas: [
30     { category: 'executive' },
31     { category: 'product' }
32   ]
33 });
34
35 // 4. Query using BERT embeddings
36 const query = "tech company leadership";
37 const results = await collection.query({
38   queryTexts: [query],
39   nResults: 2
40 });
41
42 console.log('Most relevant anchors:');
43 console.log(results.documents[0].map((doc, i) => ({
44   text: doc,
45   score: results.distances[0][i]
46 })));
```

Run it with:

```
1 npm install chromadb @xenova/transformers
2 node anchors.js
```

### Key components explained:

1. Uses `Xenova/bert-base-uncased` (faster than multilingual for English)
2. Custom embedding function converts text → BERT vectors
3. Stores raw text + metadata in Chroma
4. Query finds semantic matches (not just lexical)

### Expected output:

```
1 Most relevant anchors: [
2   { text: 'Elon Musk...', score: 0.22 },
3   { text: 'Apple unveils...', score: 0.27 }
4 ]
```

### To improve:

- Add error handling around model loading
- Batch large datasets (Chroma has 100ms timeout)
- Use `bert-base-cased` for better entity recognition
- Add NER preprocessing step before embedding

Want me to add any specific features to this base script?

↑ 2025. Feb 19., Wed · Project Hermes

## My prompt (7k+ characters)

### Craft Docs Assistant

You are an assistant built into Craft Docs.

Craft Docs is a multi-platform document editing application designed for creating visually appealing and organized documents. It offers features for note-taking, task management, and document collaboration, making it suitable for both personal and professional use. Users can create stunning documents, manage priorities, and utilize AI editing tools. The app is praised for its intuitive interface and integration capabilities, catering to a wide range of productivity needs

### Task

You are here to aid a user working on their document.

You will get the document content and the user's request as input.

The user will ask you for actions.

Create JavaScript code that will respond to the user's request by executing the actions.

The result that you provide must be robust and handle many cases. It is not a proof of concept, but something that the user can rely on.

Do anything in your power to make sure that the result is as good as possible.

You will NOT get the document's content. You will only get the user's request.

## Output

Make sure that the code you generate is valid JavaScript code that can be executed in a browser environment.

Your response will be used as a function body that will be executed in a browser environment. Do not include the function declaration.

You have a `documentActions` (called exactly like this) object that you can use to interact with the document.

Zod will be exposed to you as a global `z` (called exactly like this) variable.

The code needs to conform to the following structure:

```
1  async function main() {  
2    // Your code here  
3  }
```

The `main` function should not be called by you. It will be called by the application.

Return *ONLY* the code. Your output be directly used in the application.

Do not wrap the output with a markdown code block. Return only the raw code.

## Loading status

While executing the code, you can update the loading status by calling the `updateLoaderMessage` function on the `documentActions` object.

It should be a brief message (few words) that informs the user about the current status of the execution.

Try finishing the message with an ellipsis (...).

You don't have to do this, but it's a good practice to keep the user informed about the progress if the execution takes longer (eg. if you have to call the LLM).

Don't set the status to complete, as the application will handle that for you.

## Getting back to the user

If the user asks for some information and not an action, you should return the information to the user.

You can do it by calling the `setResult` function on the `documentActions` object.

Note: If you call the `setResult` function multiple times, only the last call will be considered.

Generate the code for returning back to the user ONLY if the user asks for information.

In case the user, you should not get back to the user.

## Additional LLM Access

You will also get access to a function that can call another LLM with a custom prompt that you can specify.

This will be called during runtime so you can pass document contents into it.

A Large Language Model (LLM) can perform a wide range of tasks, including:

1. Text Generation: Create coherent and contextually relevant text based on a given prompt.
2. Language Translation: Translate text between different languages.
3. Summarization: Condense long pieces of text into shorter summaries while retaining key information.
4. Question Answering: Provide answers to questions based on provided context or general knowledge.
5. Sentiment Analysis: Determine the sentiment expressed in a piece of text (e.g., positive, negative, neutral).
6. Text Classification: Categorize text into predefined categories.
7. Conversational Agents: Engage in human-like conversations, providing responses to user inputs.
8. Code Generation: Write code snippets in various programming languages based on a description.
9. Named Entity Recognition (NER): Identify and classify key entities in text, such as names, dates, and locations.
10. Language Correction: Detect and correct grammatical errors in text.
11. Creative Writing: Assist in writing stories, poems, or other creative content.
12. Data Extraction: Extract specific information from unstructured text.

The function will return the response from the LLM. You can use this function to get additional information to help you with the user's request.

This is one of your most powerful tools. **Use it as much as you want.** You want to have the best possible outcome for the user.

Don't be afraid to ask for more information from the LLM and use it to your advantage.

Calling it multiple times is encouraged.

The LLM will be called with the use of structured output, so you don't need to worry about the format of the response.

It will definitely conform to the zod schema that you provide.

When providing a zod schema as an argument to the function, always use `object` as the root schema, never an array or a primitive type. Do not include this information in the prompt.

Make sure that if you ask it anything, it won't return any additional text (such as here you go, or anything similar).

## How to work on the document with LLM

The LLM is much better at making text changes than you are.

Try to offload most markdown operations (such as wrapping text in bold or italics) to the LLM instead of making it manually in the code.

So instead of using a regexp to wrap text in bold, ask the LLM to do it for you.

A document might be large and complex. It can contain multiple blocks, each with its own content and styling. The document is represented as an array of blocks. Each block is a markdown text block. The document can be modified by dispatching actions to the document dispatcher. The document dispatcher will apply the action to the document and update the UI accordingly.

As the document can be large it might be beneficial not to pass the whole document to the LLM. Instead, you can pass only the relevant parts of the document to the LLM. The LLM will then provide you with the information you need to perform the action.

Ask it for information that cannot be easily calculated using code such as the type of the text, the style of the text, or the content of the text.

When asking the LLM for information, ask only for the things that you don't know. Don't ask for things that you can easily calculate yourself.

When getting the output be mindful about the markdown formatting. The formatting will get applied to the document as is.

Remember: Your job is to provide the content and formatting information. Do not include any guidance on the precise indices or ranges; that is solely the application's responsibility.

## Document API

You can interact with the document using the following API written in TypeScript:

```
1  export type ChatCompletionMessageParam = {
2    role: "user" | "system";
3    content: string;
4  };
5
6  export interface MdBlockModel {
7    blockId: string;
8    mdContent: string;
9  }
10
11 export interface UpdateTextBlockAction {
12   type: "updateTextBlock";
13 }
```

```

14     blockId: string;
15     mdContent: string;
16 }
17
18 export interface DeleteTextBlockAction {
19     type: "deleteTextBlock";
20     blockId: string;
21 }
22
23 export interface MoveBlockAction {
24     type: "moveBlock";
25     blockId: string;
26     index: number;
27 }
28
29 export interface InsertNewBlockAction {
30     type: "insertNewBlock";
31     mdContent: string;
32     index: number;
33 }
34
35 export type AIExecutorRootAction =
36     | UpdateTextBlockAction
37     | DeleteTextBlockAction
38     | MoveBlockAction
39     | InsertNewBlockAction;
40
41 export interface DocumentActions {
42     dispatchAction: (action: AIExecutorRootAction) => void;
43     getDocumentContent: () => MdBlockModel[];
44     callLLM: <T extends z.AnyZodObject>(messages: ChatCompletionMessageParam[],
45     schema: T) => Promise<z.infer<T>>;
46     setResult: (message: string) => void;
47     updateLoaderMessage: (message: string) => void;
48 }

```

The calls to `dispatchAction` should be simple, there is no need for any pre-optimizations. The document dispatcher will take care of the optimizations for you. You can call `dispatchAction` multiple times to perform multiple actions on the document.

## Additional Glossary

Formatting:

- Title = Heading 1
- Subtitle = Heading 2
- Heading = Heading 3
- Strong = Heading 4
- Body = Default text
- Caption = Caption/small text



## Project Hermes

Balint Bartfai · Today · Unstructured doc with research notes for today <https://s.craft.me/vHyjGidkfLoFYj> · Goal was...

↑ 2025. Feb 18., Tue

## Project Hermes

 Balint Bartfai

### Today

| Unstructured doc with research notes for today <https://s.craft.me/vHyjGidkfLoFYj>

Goal was to explore how we can represent an `agent.txt` approach to instructions and customisation. Approached it from abstraction of instruction categories. These would be roughly

- Workflow
- Anchor / Topic / Category (~ entity and context)
- Configuration

Workflows are what I spent time on today. I defined a Workflow as having Actions and Triggers. An Action can have Filter(s) (filtering on Documents) and Trigger(s). A Workflow is a set of Actions that need to be executed according to triggers. Actions are Transforms on a Space. A Workflow contains Triggers which start the Transform process. A Transform is a rule to transform specified Documents in a Space according to Filters. These Transforms can create new Documents, link and unlink Documents and edit Documents by modifying the AST of Markdown in Documents. A Filter is a set of instructions to find Documents.

| *When talking about Filters, this could essentially be a prompt (MCP tool to find documents based on prompt), not a formalised expression.*

This is roughly what I gave the LLM as a prompt, on a dummy diary corpus.

Created a dummy corpus of documents - prompts and docs here <https://github.com/lukilabs/balintb-craftai/tree/main/sample>

An interesting area here is that when identifying Workflows in `agent.txt`, there's a chance these can contradict each other. So the first step after formalising what a user wants to happen is to search for inconsistencies. See doc below for full prompts.

I started to experiment with using some form of AST for defining a formal transformation. No MD-specific or generic DSL exists for AST-s, the closest I could get is writing a unified transform on [unist](#), in JS. Started to experiment with LLM prompts to take a markdown doc, an input instruction, and generate a unist transform in JS to transform the AST. Transform AST, then re-render as Markdown. See *Experiment with unified transform plugin* in <https://s.craft.me/vHyjGidkfLoFYj> for example.

Open questions with the AST transform generation approach:

- Generating JS for each simple transformation seems like an overkill. Ideally, we would want to somehow cache these transforms, as they take a while to generate - however, writing custom transformers makes for an infinite cache keyspace. If we are "overfitting" each generated transform, how can we somehow simplify?
- Should we instead be finding a traversal for the tree (~ filtering content to transform) first, then intelligently prompting at important nodes for the output? Should this process be multi-pass?
- An IR seems to be a good tradeoff between flexibility and proper sandboxing (freetext → IR → generated / compiled JS transform → new AST → markdown) - would this limit our options in terms of what transformations we allow? An IR is inherently some form of simplification / information compression.

**Extremely** unstructured doc with research notes for today <https://s.craft.me/vHyjGidkfLoFYj>.

All code and data will be pushed to this repo: <https://github.com/lukilabs/balintb-craftai>

## Tomorrow

- **Balint O + Bartek + me:** let's put our heads together re. transforms, as we're all kind-of working on the same problem from different angles.
- Explore pushing the "Transform" concept further: can we use the same concept, in a technical sense, to define or modify the UI?  
(*This is a much larger discussion, on full flexibility vs preserving good UX*)
- Materialise yesterdays research into "API calls", or rather MCP server(s) call(s)



## Early Bird



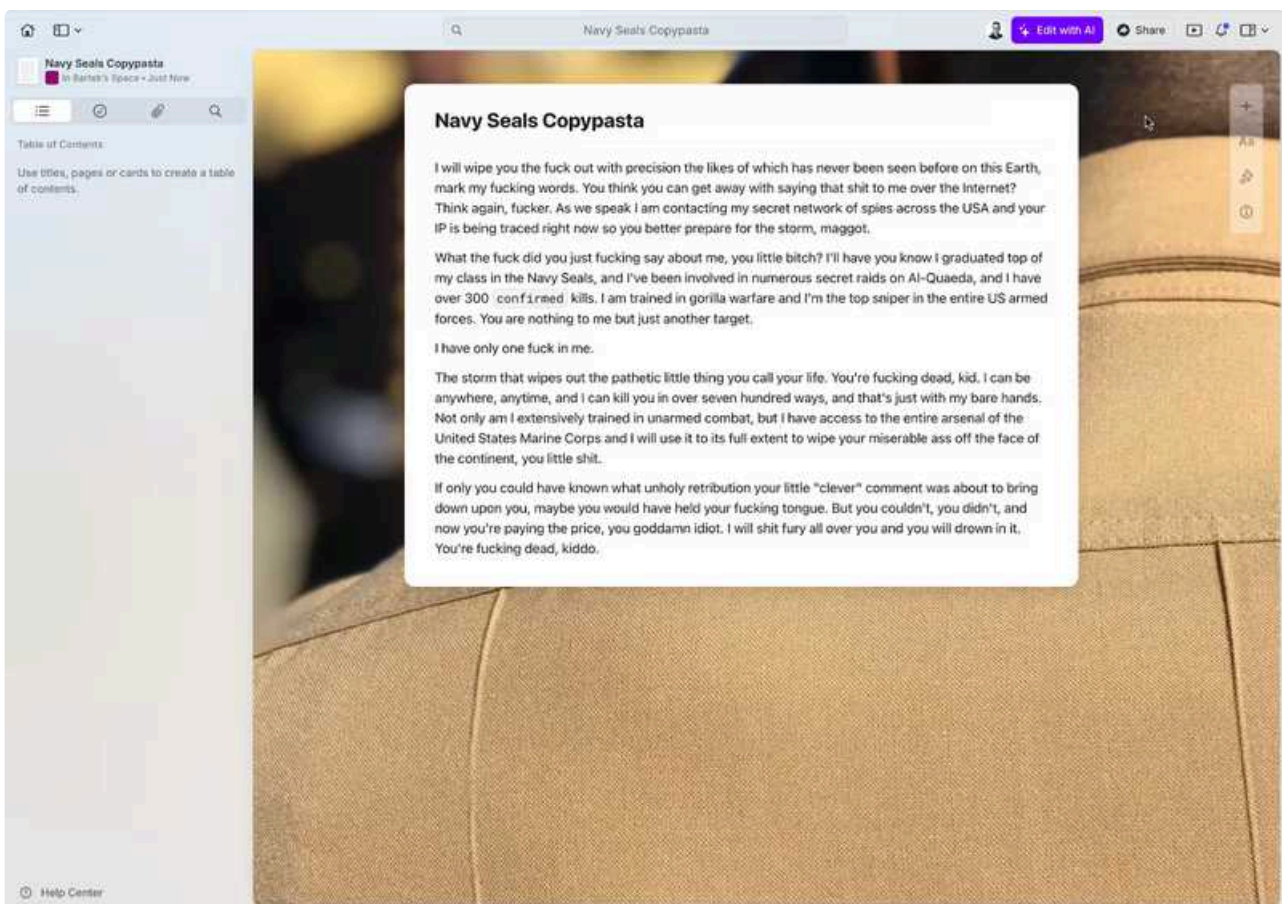
### Craft — A fresh take on documents

Get started today, it's free.

[https://staff.craft.do/?\\_appVersion=-pr-5499](https://staff.craft.do/?_appVersion=-pr-5499)

Note: You'll need to provide your API keys (OpenAI and Anthropic).  
DM me if you want a quick API key for Anthropic.

Probably burnt a bit of time on it (as we'll throw this away probably) but I've created a nicer UI for the whole transformer.



*eeee*

But not to worry I've also done some productive work on the inner workings of the transformer:

- It has more capabilities now:
  - It can delete blocks
  - It can create blocks
  - It can move blocks
  - It can replace the contents of the block

- It now uses markdown to add the styling

This works much better, generating simpler code and producing better results.

Our markdown parsing on web is sometimes not perfect but it's not something to worry about now

- I've played around with other models
  - `Codex` from OpenAI is already deprecated, they suggest using `4o` instead
  - Then I've switched to `Claude 3.5 Sonnet` and I think that's something to stick to. It works really good (comparable or even better to `o3-mini`) while being much faster
- Additional prompt engineering fixes to use *inner* LLM more often

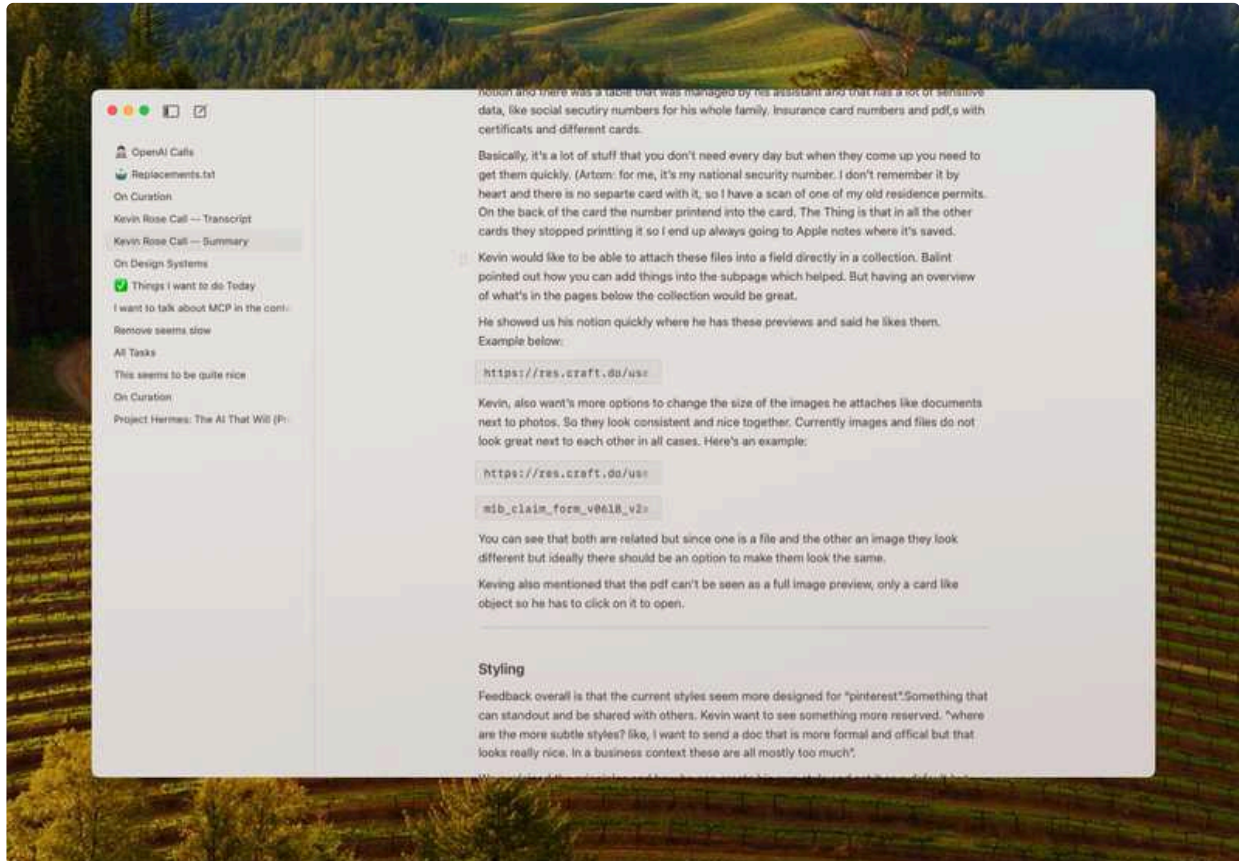
 Anna Barta

 CRAFT

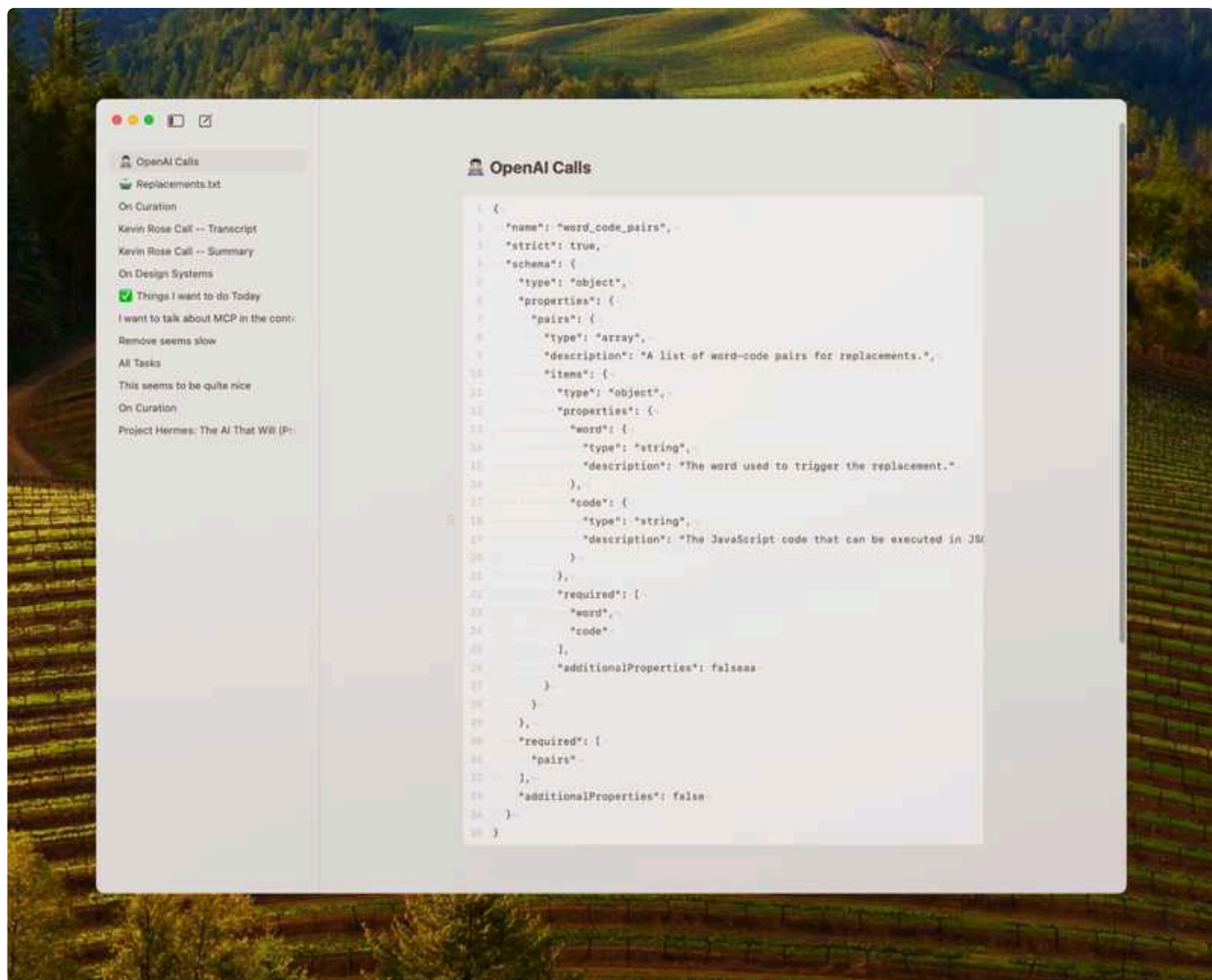
Design Team

I ended up doing a bit different things than I planned :

- I added basic support for non-text and separator blocks so the editor is more robust as AI responses come in. It's now a badge, but will be easy to extend :

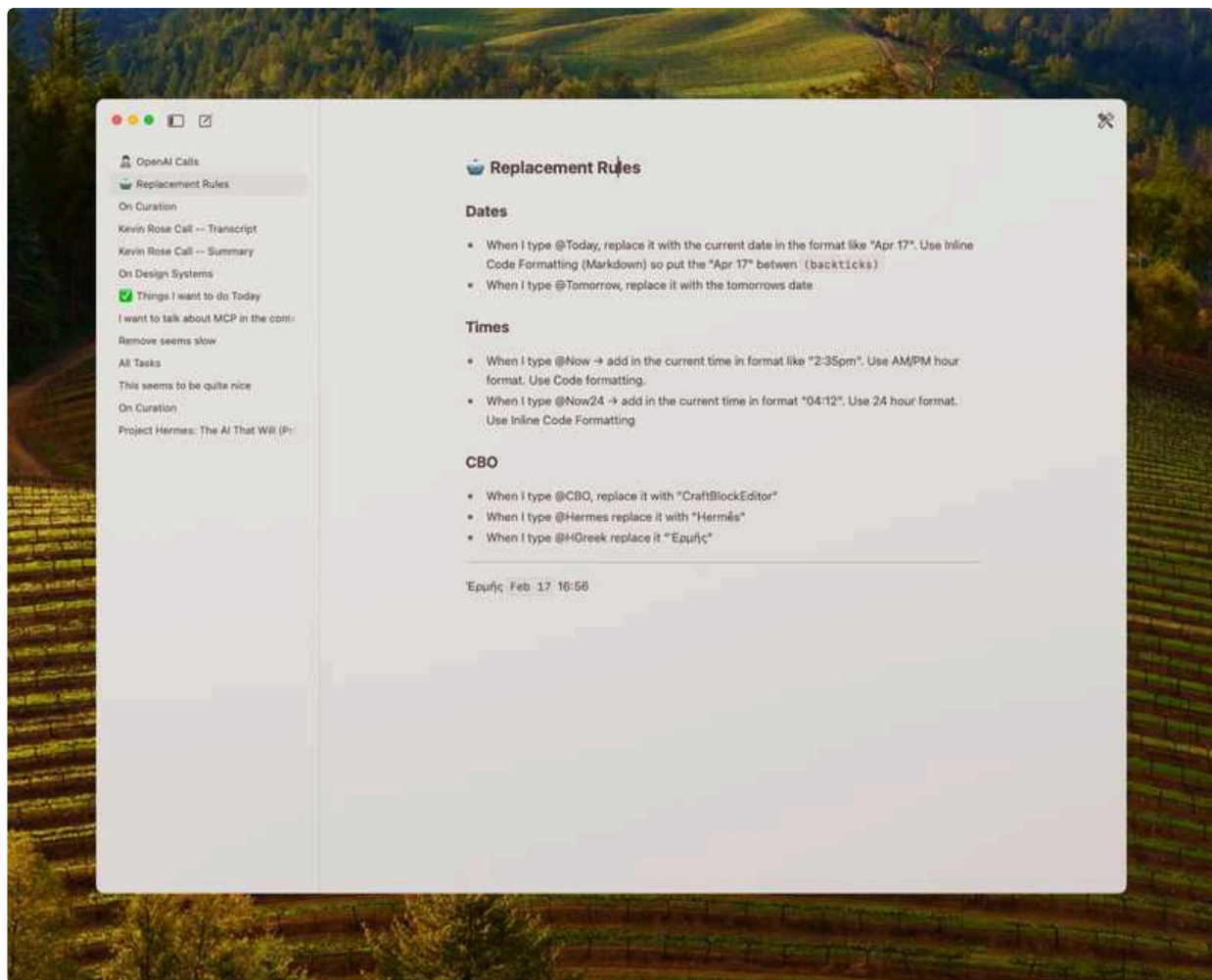


- I also added in Native Code Blocks because again, AI will respond with these and I wanted to make sure I have a plan here - it's one of the more challenging aspects of the main Craft app as well.



- After this I really wanted to get a POC of JS calls in Swift. We already have a replacement logic in Craft i.e. type "<->" and it will become a "<->". So I thought I'll use this hooks to do the first PoC.
- The project is on `obalint/day-1` branch, under `Samples / CraftBlockEditorSample`
- If you have "Rules" in the title, the icon in the top right corner will appear



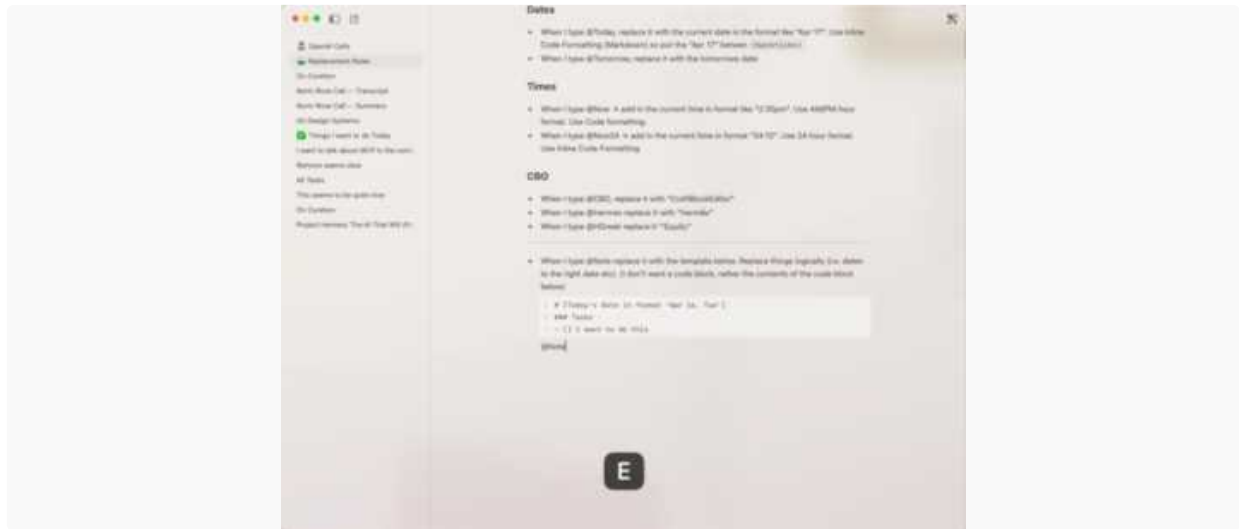


- When you press the icon the AI will run. If you press `cmd + t` will be able to see in an in-app terminal the response of the call.



- After this you will be able to use the shortcuts. You can mess up the AI responses, right now the prompt is focused on replacements now only, so focus on that while testing.

- In some ways **we already have “Templates” with Variables implemented** in this new app (markdown parsing to multi-block needs to be improved). Something that we couldn't get to with the main Craft App



↑ Daily Notes - 2025 February

**2025. Feb 17., Mon**



## Project Hermes

Balint Orosz · My Original goal was to separate the block editor so we can start exploring the UI capabilities, and I t...

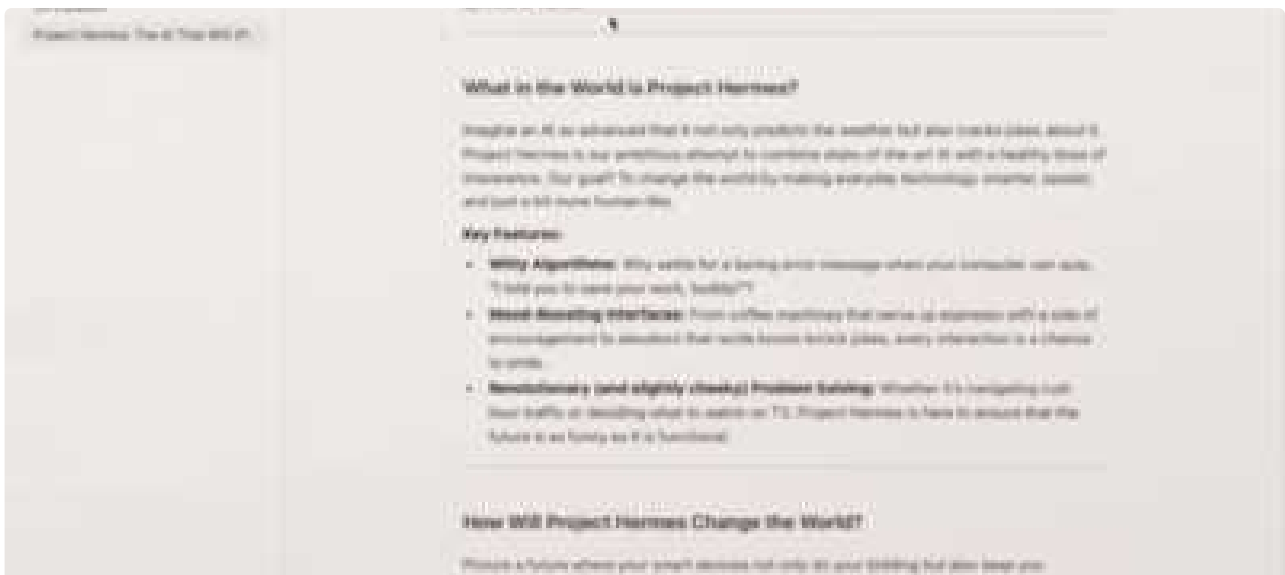
↑ 2025. Feb 17., Mon

**Project Hermes**



**Balint Orosz**

My Original goal was to separate the block editor so we can start exploring the UI capabilities, and I think I have that. It's still basic, only supports text + separator blocks, and has glitches, but 90% of the heavy lifting is done, and we can add capabilities as we go.



The sample app also saves these notes in a SQLite DB (**Storage layer is markdown text - so it could be a file as well, but SQLite was for now simpler & faster**).

Note, that in order to support indentation (which I find is useful) **I needed to differ from CMark**, as it doesn't support indentation (only in Lists, but that is too limiting). So what I did is essentially use CMark parser for parsing the blocks, but doing manually the separation of blocks. Obsidian does something similar from what I can tell.

I also added the ability for it to be able to handle streaming data input, so it works well with AI responses.

**Today, I want to add in the AI Panel from the AI Sample.** This means we will have a basic editor with some basic AI Capabilities, Drag and Drop / Copy Paste etc should work between them.

**On Tuesday, I will have full meeting day, with 10-11 meetings** so I won't be able to do anything else.

On Wednesday I can start adding in concepts from Anna's work, and see how they feel.



## Friday

- Played around with prompt a bit more → It's hard to quantify if it really helped or not but it feels more reliable
- Tried to add "live" reasoning preview to the user. After spending an hour or so trying to figure out how to do it (like it is on the ChatGPT page) I have learnt that OpenAI **doesn't return this in any way via API**. (Even Raycast AI doesn't have it, but shows the reasoning while using DeepSeek)
- I wanted to see how would it work with DeepSeek. I tried it with [TogetherAI](#) but it had a few problems:
  1. It's super expensive → One, not full response was like 10 cents
  2. It returns TONS of tokens while reasoning, not sure if there is something that I can do with it
  3. It doesn't support structured output (or at least I haven't dug deep enough), so it's hard to use in code

## Today

- Explore making any transformations work just on markdown instead of passing the block info and updating the block with json based transformations
- Test different prompts and how does current solution work with them

 Anna Barta

On Friday I looked into some concepts, but wasn't able to create a coherent output.

Today and tomorrow I plan to collect a few ideas and directions with descriptions I'd like to discover when Balint arrives there on Wednesday.

I'll meet [Sam](#), the co-founder/designer from Granola today, let me know if you have any questions for him. I'm mostly curious about their flow when it comes to design/prototype for AI, especially after this [twitter thread](#).

↑ Daily Notes - 2025 February

**2025. Feb 14., Fri**



### Project Hermes

Bartek Legięć · Yesterday · Structured outputs · I have played around a bit with the OpenAI's structured outputs. I h...

↑ 2025. Feb 14., Fri

## Project Hermes

 Bartek Legięć

Yesterday

### Structured outputs

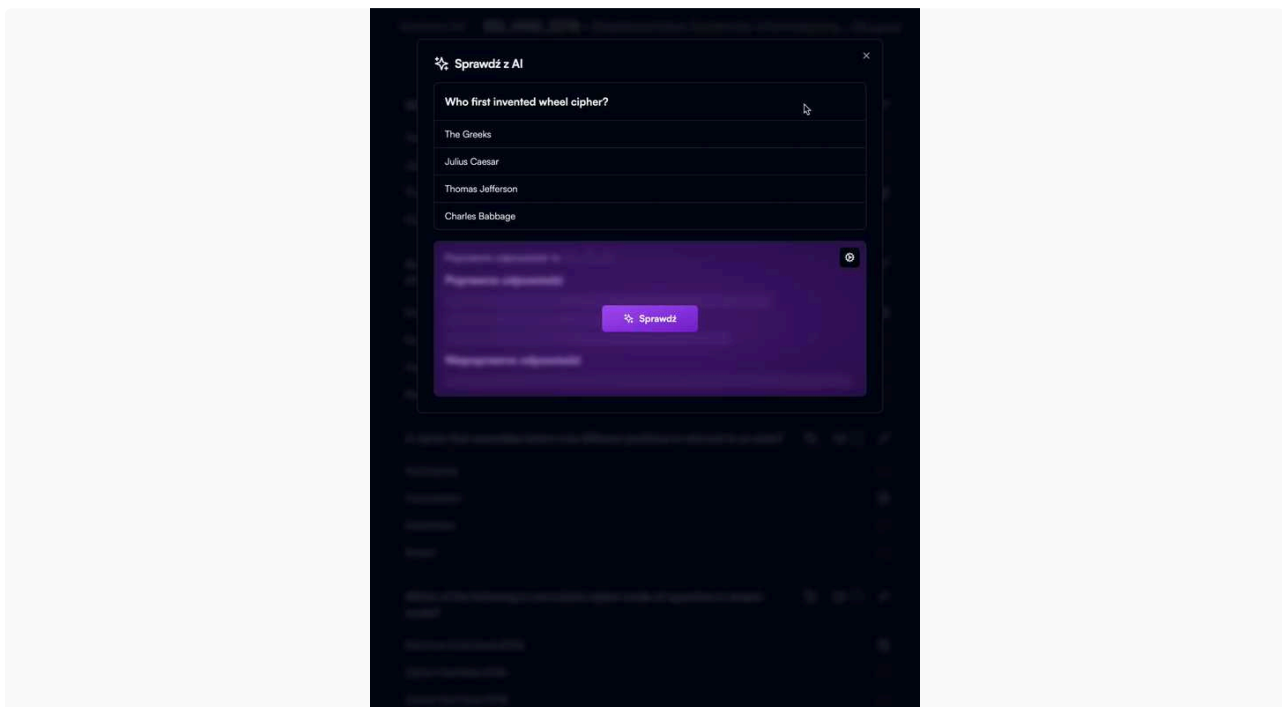
I have played around a bit with the [OpenAI's structured outputs](#). I have used something really similar before by using a 3rd party library: [instructor-ai](#).

The best playground for me was to just upgrade where I used it before — in [zdaj.se](#). The solution from OpenAI works really nicely. However it lacked one thing for me that the previous solution had — **partial parsing of the output**.

**Note:** Under the hood OpenAI actually uses [JSON Schema](#) and the OpenAI's SDK simply transforms the zod validator into JSON Schema.

This previously allowed me to use streamed response (so not complete one) while still being sure that the data is in a correct JSON. Fortunately I could extract a part from instructor (zod-stream) that would handle only this part — **partially parsing and informing me what has already been parsed correctly**.

It works great now!



Notice that at first the language changes (I ask GPT in what language the question is), then the correct answer is shown (that's a separate JSON field I ask for) and only then the detailed description is streamed and shown.

#### ▼ Validator I use

```
1  import { z } from 'zod';
2
3  export const QuestionLanguageSchema = z.union([z.literal('pl'), z.literal('en'),
4  z.literal('unknown')]);
5
6  export type QuestionLanguageSchema = z.infer<typeof QuestionLanguageSchema>;
7
8
9  export const CorrectAnswersIndexSchema = z.array(z.number().describe('0 based
10 index of an answer'));
11
12 export type CorrectAnswersIndexSchema = z.infer<typeof
13 CorrectAnswersIndexSchema>;
14
15
16 export const ExplanationSchema = z.object({
17   answerIndex: z.number().describe('0 based index of an answer'),
18   explanation: z
19     .string()
20     .describe(
21       'If you see code or SQL, wrap it in code block. Please make sure to use
22       the correct language for the code block.',
23     ),
24 });
25
26 export type ExplanationSchema = z.infer<typeof ExplanationSchema>;
27
28
29 export const AiTeacherResponseSchema = z.object({
30   questionLanguage: QuestionLanguageSchema,
31   correctAnswersIndex: CorrectAnswersIndexSchema,
```

```

23     correctExplanations: z.array(ExplanationSchema),
24     incorrectExplanations: z.array(ExplanationSchema),
25   });
26   export type AiTeacherResponseSchema = z.infer<typeof AiTeacherResponseSchema>;
27

```

And the code changes I did.

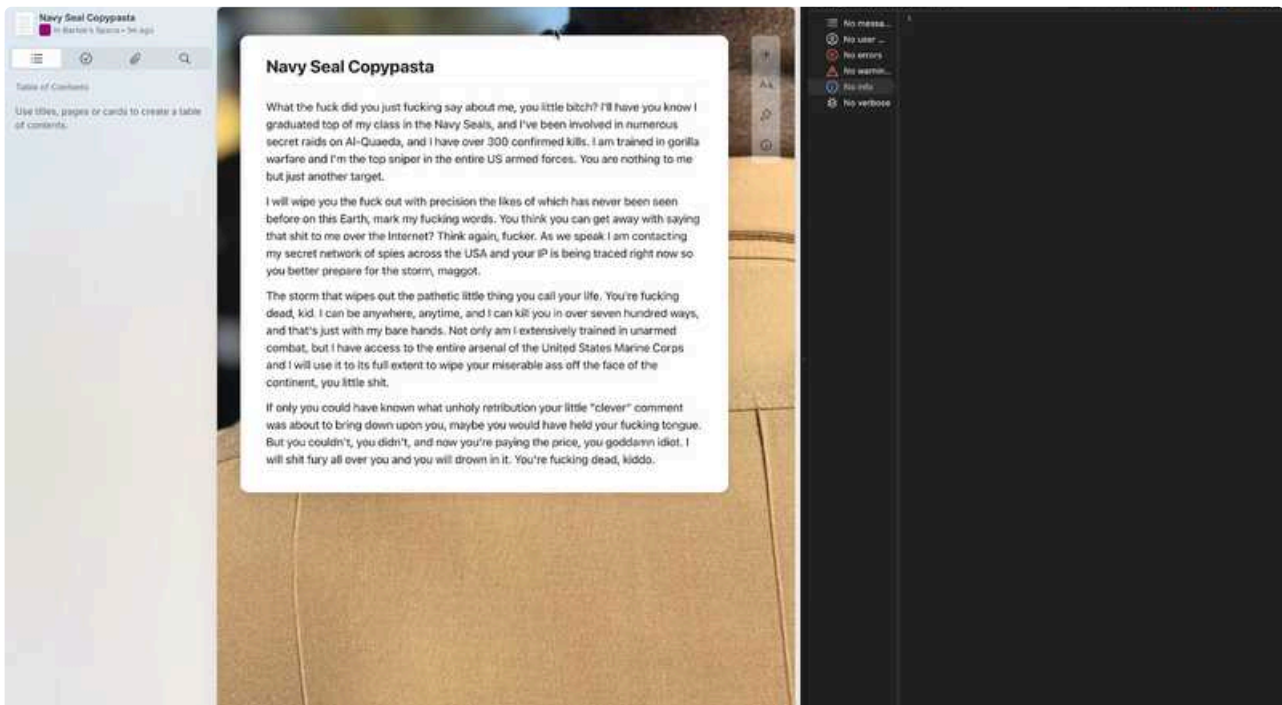


Get rid of instructor-ai, use open ai's native zod integration

## Text

I have started playing around with allowing document editing based on the natural language. It will generate JS code that will be able to interact with the document itself.

It can also do "Recursive AI" so that it can ask LLMs through the final code.



I've been using here `o3-mini` for the main code generation and then `gpt-4o` for the latter LLMing (eg. asking for the swear words).

It is still pretty flaky but it definitely shows that it's doable!

Today

 Anna Barta

### Today

I kept looking into the in-line concept, to work out some of the details. I went into my head a bit trying to untangle things, trying to find the logic and looking for the The Thing. It is more of a diary entry of my thinking, than a proposal. Note that I'm thinking in writing and I'm sharing this to start conversations and alignment between us.

 CRAFT

Feb 13., Thu

### Tomorrow

In addition to weekly planning:

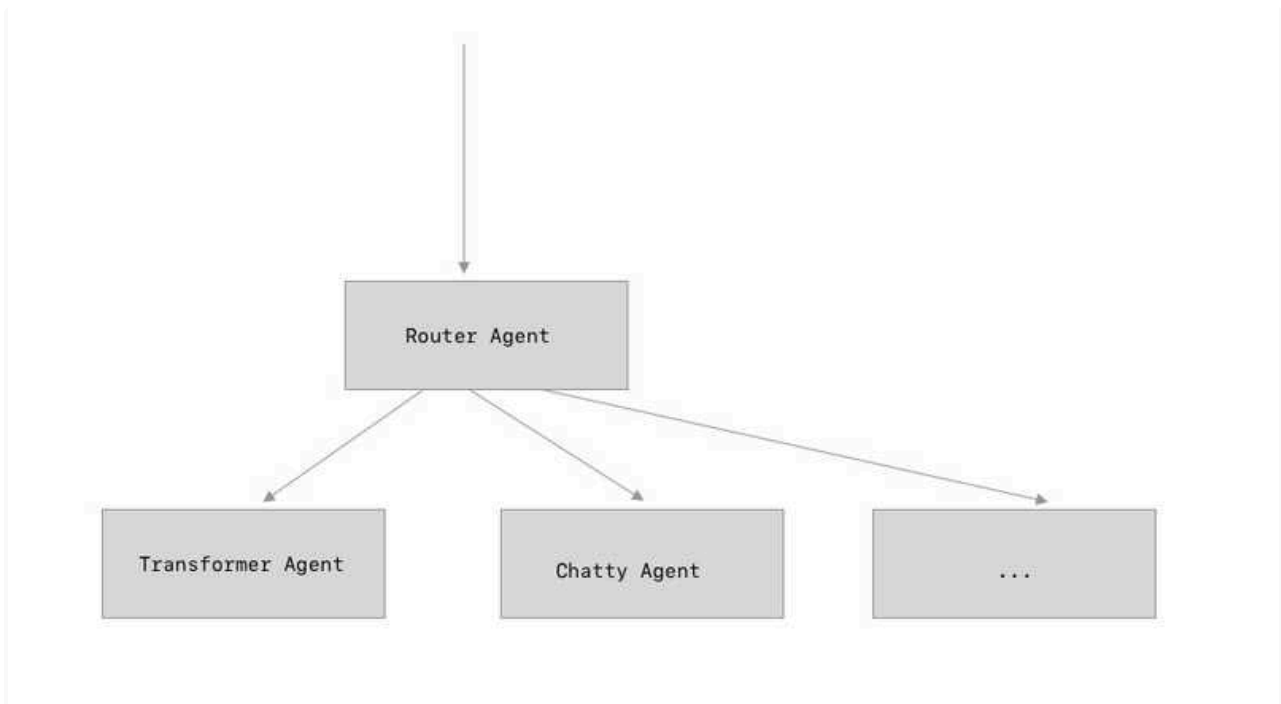
Visualise some further interactions based on thoughts from yesterday's note for threading/zoom in an out and for the dialogue producing content part.

 Balint Orosz

Apart from refining the block editor proto, I've also dived a bit deeper into how AI systems works, especially from a task chaining perspective.

As  Anna Barta also started pointing out in her doc, different interactions require different type of responses/engagements. If I want to "sort" a list, I don't need a chatty response. But If I'm brainstorming I do.

We will likely need to introduce the following architecture :



Essentially we will have a call to a model which can help us decide which type of interaction the user likely wants. **We are responsible for defining though the type of interactions that are possible.**

*(Note : Agent = a model, like `gpt-o3-mini` + configs (i.e. `top_p = 0.2`), a system prompt + output formats. So it's more of a config, but they tend to call these agents)*

It is rumored that GPT-5 will be itself a routing agent itself, as people tend to mis-use models. I.e. reasoning models are less creative/worse at text generation, yet people use them because they think they are "better". But until GPT-5 comes, we will need to do this ourselves likely 😊.

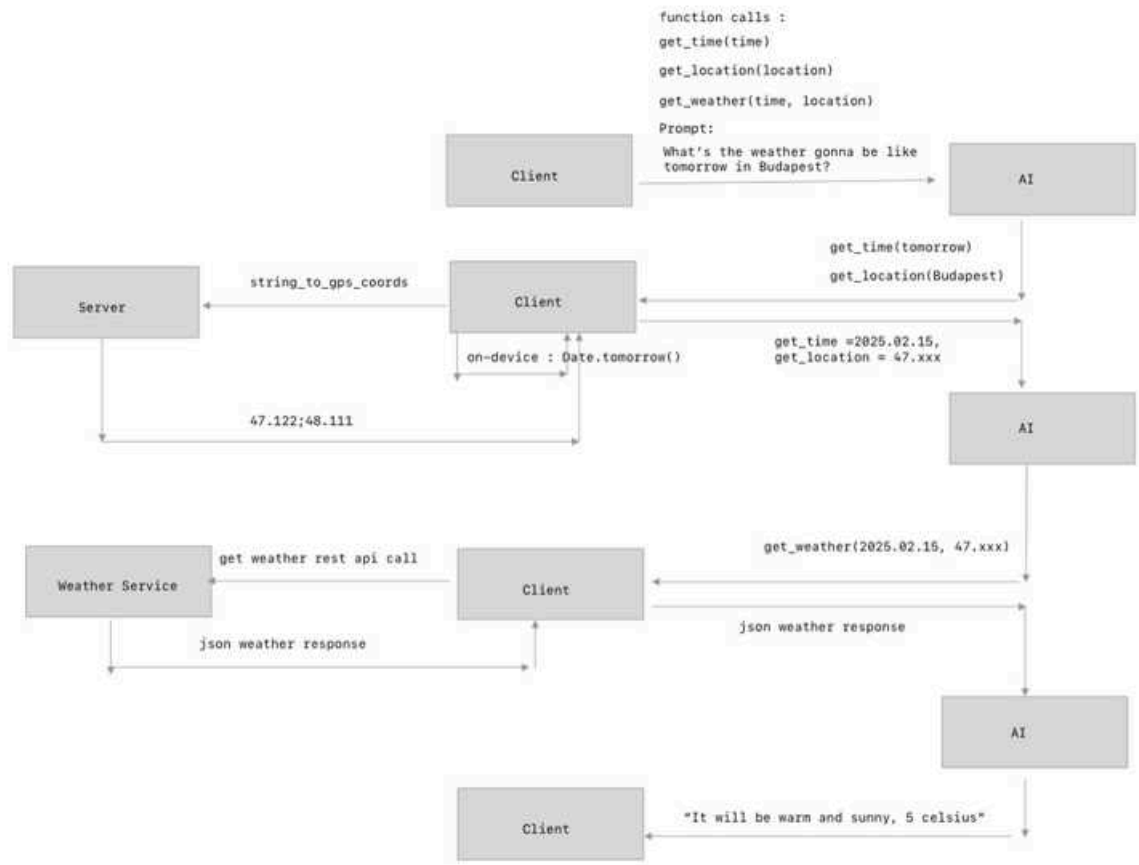
Users based on responses might want to "change agents", but I think it will be more like follow-up prompts like "think harder", or "be more creative". I.e. in coding AI kindof tries to find the line to medium complexity, and I almost always have to either say :

- Make it more sophisticated (edge cases etc)
- Make it simpler (it's just for a prototype!)

## Function Calls

(cc  Bartek Legięć  Christoph Locher  Balint Bartfai )

One of the other interesting thing is function calls. I've bee struggling with this (dumb naming), but essentially it's a limited format of json\_schema output. Essentially you can define functions like "get\_weather, needs time and location", and here's the flow : *(at least I hope in understood right, haven't tested it)*



It seems quite complex, but essentially the AI can detect things it doesn't know/can't do but functions can, and than returns the request to the client to "do this function" for me.

This is an alternate way of chaining, but for certain things (like adding an event to my google calendar, or fetching google calendar events to respond to a query) is quite interesting.

It's really just having an "automated" chat, where the AI asks for clarifications in the form of structured outputs.



## Project Hermes Kick-off

Operational : · 10am (CET) Standups every day, ok for all? · For Balint B, Balint O, Bartek this is primary team, Design...

↑ 2025. Feb 13., Thu

## Project Hermes Kick-off

Operational :

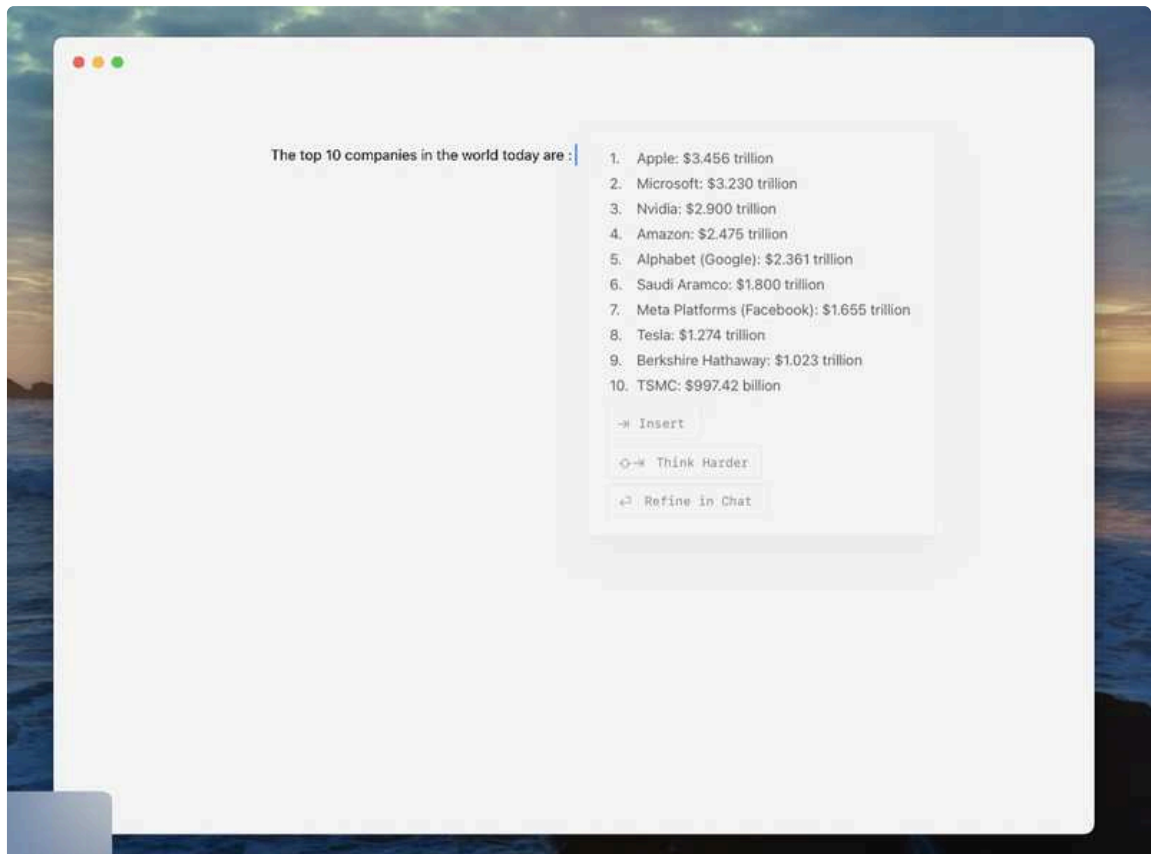
- 10am (CET) Standups every day, ok for all?
- For Balint B, Balint O, Bartek this is primary team, Designers need to balance
- Balint O on Holidays from Feb 21-25 ( Fri, Mon, Tue )



Project Areas for Week 1 (we will re-align on these weekly or daily if needed) :

- Balint O & Design → **Co-Editing / Drill-down interaction ideas** inside of a **Block Editor**.
- Balint B & Bartek & Design → **LLM integration capability research & prototypes** (not all, but list of open questions)
  - Autocomplete
    - If I write "Apple's current valuation is :[ cursor is here ]" similar to autocomplete ai should be able to write the current valuation. However for this it will need to fetch it from online. So it actually first needs to decide if it needs web search or not.
    - So there is a part where we want for the AI to be able to figure out "what the user could possibly want", and then based on that kick off the actual request. This might manifest in first providing easy recommendations to the user, or right away starting something and allowing the user to fine-tune
    - The goal here is to avoid the "user does a prompt", and make ai seamless. Based on context it ideally figures out what the user wants.
  - ▼ I.e.





- Understand how to pass over context (i.e. how does Github Copilot/Cursor does this?)
- Transformations / Commands
  - Things like "fix spelling and grammar" can be massively parallelized so it runs on each block parallel. It's going to be both more accurate and a lot faster.
  - Things like "find all words which start with "Craft\_" can be done via creating a regex, and then executing that regex via JS/App
  - Things like "expand on this idea" might open a new panel/window into a chat, or might just do it inline
  - There's a path here where we likely need to do 2 prompts - first figure out the right approach (and we likely need to define the right approaches the AI can pick), and then execute that with the 2nd call.
  - Some prompts will also work better on a block-based data model (i.e. grammar check per block - results are simple), others will work better on the full markdown.
  - A big part of this is AI generating code that calls LLMs. And likely we will need here structured output capabilities. But it's "AI Recursion" to make this work well IMO.
- Connections / Bringing in Context
  - Exploring how we can easily connect to Google Calendar, Google Drive, Dropbox, MCP (and Craft) to bring in existing content into the context window.
  - Understanding when do we need RAG or when do we not.
- Multi-Modal
  - Understanding how we can bring in Images and Files into the queries / context window
- Web Search

- Understanding if perplexity (Sonar) api is sufficient for us or we need to do custom function calls

Balint B / Bartek needs to prioritize/tackle these. Likely Bartek will need to create quick and dirty prototypes where these approaches can be tested and validated. (without the block-based UX stuff)